

Clean Core: AI - Driven Waste Monitoring & Optimization

Ayesha Noman

Ayesha Nadeem



**Department of Computer Sciences
Comsats University Islamabad, Wah
Campus Wah Cantt-Pakistan**

Session 2022 – 2026

Clean Core: AI - Driven Waste Monitoring & Optimization

Undertaken By:

AYESHA NOMAN

REG. NO. CIIT/FA22-BCS-009/WAH

AYESHA NADEEM

REG. NO. CIIT/FA22-BCS-020/WAH

Supervised By:

Ms. MAHA RASHEED



A DISSERTATION SUBMITTED AS A PARTIAL FULFILLMENT OF THE
REQUIREMENTS FOR THE DEGREE OF BACHELOR'S IN COMPUTER SCIENCE

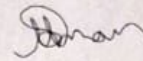
**DEPARTMENT OF COMPUTER SCIENCES
COMSATS UNIVERSITY ISLAMABAD, WAH CAMPUS
WAH CANTT - PAKISTAN**

SESSION 2022-2026

Final Approval

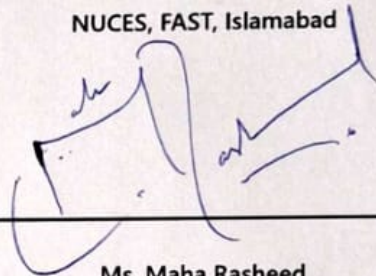
Certified that the project report titled, "CleanCore – AI-Driven Waste Monitoring and Optimization", completed by AYESHA NOMAN (CIIT/FA22-BCS-009/WAH) and AYESHA NADEEM (CIIT/FA22-BCS-020/WAH) is of sufficient standard, in our judgment, to warrant its acceptance by COMSATS University Islamabad, Wah Campus for the award of Degree of Bachelor of Science in Computer Science / Software Engineering.

1. External Examiner Name



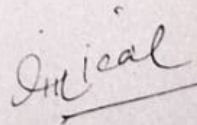
Dr. Usman Habib
Professor
NUCES, FAST, Islamabad

2. Supervisor



Ms. Maha Rasheed
Lecturer
CUI Wah Campus

3. Head of Department



Dr. Muhammad Khalil Afzal
Head of Department
CUI Wah Campus

DEDICATION

We begin with the name of Allah, the Most Gracious, the Most Merciful. With His endless blessings, guidance, and strength, we were able to walk through this journey with determination, clarity, and gratitude.

This project is wholeheartedly dedicated to our beloved parents, whose love, prayers, and unwavering support have been the greatest sources of strength for both of us. Their sacrifices, encouragement, and constant presence in our lives kept us motivated, especially in moments when the path felt overwhelming. Whatever we achieve reflects their guidance and unconditional belief in us.

We also extend this dedication to our teachers and mentors, whose encouragement and direction have shaped our learning throughout our academic journey. Their support, both inside and outside the classroom, has played a vital role in helping us reach this milestone.

Lastly, we dedicate this work to the Almighty Allah once again whose mercy, protection, and blessings enabled us to complete this project with patience, perseverance, and a sense of fulfillment. Every step of this achievement belongs to Him.

Ayesha Noman

Ayesha Nadeem

ACKNOWLEDGEMENT

Gratitude and endless thanks to Allah Almighty, the Lord of the Worlds, who has bestowed upon humankind the light of knowledge through perception, learning, and reasoning, guiding us in our search for truth. We Honor the Prophet Muhammad (Peace Be Upon Him), whose teachings continue to illuminate our path. We are deeply indebted to our beloved parents, whose unconditional love, sacrifices, and prayers have made our achievements possible. Our heartfelt gratitude extends to our teachers, especially our respected supervisor, Ms. Maha Rasheed, for her invaluable mentorship, unwavering support, and encouragement. Her guidance, motivation, and wisdom have not only helped us refine our project ideas but also inspired us to work diligently and present our work in the best feasible way.

PROJECT BRIEF

PROJECT NAME	CLEAN CORE
NAME	COMSATS UNIVERSITY ISLAMABAD, WAH CAMPUS
OBJECTIVE	TO DEVELOP AN AI-POWERED SMART WASTE COLLECTION SYSTEM THAT ENABLES DRIVERS TO VIEW BIN FILL LEVELS AND RECEIVE OPTIMIZED COLLECTION ROUTES USING COMPUTER VISION BASED PREDICTIONS AND MULTI OBJECTIVE ROUTE OPTIMIZATION.
UNDERTAKEN BY	AYESHA NOMAN AYESHA NADEEM
SUPERVISED BY	MS. MAHA RASHEED LECTURER COMPUTER SCIENCE COMSATS UNIVERSITY ISLAMABAD, WAH CAMPUS
STARTED ON	OCTOBER 2025
COMPLETED ON	MAY 2025
COMPUTER USED	HP PROBOOK 450 G7 – INTEL CORE i5-10210U CPU, 8G RAM, NVIDIA MX130 GPU
SOURCE LANGUAGE	DART, FLUTTER, PYTHON (FOR AI TRAINING)
OPERATING SYSTEM	WINDOWS 10 PRO
TOOLS USED	VISUAL STUDIO CODE

ABSTRACT

Clean Core is a mobile based smart waste collection system designed to transform urban waste management by combining AI powered bin monitoring with intelligent route optimization. The platform enables waste collection drivers to view assigned bins with predicted fill levels, receive optimized collection routes that balance urgency and distance, manage profiles, and navigate using integrated maps. Bin fill levels are predicted using a YOLO based image classification model trained on waste bin imagery. An administrative panel allows monitoring of bins, assignment of areas to drivers, scheduling of time slots, and management of the overall collection workflow. Developed using Flutter, Firebase, React, and Python, Clean Core provides an efficient, data driven approach to waste collection that reduces operational costs, prevents bin overflow, and promotes smarter city sanitation.

TABLE OF CONTENTS

1	INTRODUCTION	1
1.1	SYSTEM INTRODUCTION.....	1
1.2	BACKGROUND OF THE SYSTEM.....	1
1.3	OBJECTIVES OF THE SYSTEM.....	2
1.4	SIGNIFICANCE OF THE SYSTEM.....	3
2	REQUIREMENT SPECIFICATIONS.....	4
2.1	PRODUCT SCOPE	4
2.2	PRODUCT DESCRIPTION	4
2.2.1	Product Perspective	4
2.2.2	Product Functionality	5
2.2.3	Users and Characteristics	6
2.2.4	Operating Environment	7
2.3	SPECIFIC REQUIREMENTS.....	8
2.3.1	Functional Requirements.....	8
2.3.2	Behaviour Requirements	11
2.3.3	External Interface Requirements	19
2.4	NON-FUNCTIONAL REQUIREMENTS	24
2.4.1	Performance Requirements.....	24
2.4.2	Safety and Security Requirements.....	25
2.4.3	Software Quality Attributes.....	25
3	DESIGN SPECIFICATIONS	27
3.1	INTRODUCTION	27
3.2	COMPOSITE VIEWPOINT	27
3.2.1	Package Diagram:.....	27
3.2.2	Deployment Diagram (Physical)	29
3.3	LOGICAL VIEWPOINT	30
3.4	INTERACTION VIEWPOINT	31
3.4.1	Sequence Diagram for Admin Manage Users	31
3.4.2	Sequence Diagram for Admin Manage Bins	32
3.4.3	Sequence Diagram for Admin View Dashboard	33
3.4.4	Sequence Diagram for Admin Generate Reports	34
3.4.5	Sequence Diagram for Admin Login.....	35
3.4.6	Sequence Diagram for Worker Login.....	36
3.4.7	Sequence Diagram for Worker Register.....	37
3.4.8	Sequence Diagram for Worker View Bin Status	38
3.4.9	Sequence Diagram for Worker Plan Optimized Route.....	39
3.4.10	Sequence Diagram for Worker Receive Alerts.....	40
3.4.11	Sequence Diagram for Worker Mark Bin as Collected	41
3.4.12	Sequence Diagram for Worker Report Bin Anomaly	42
3.5	STATE DYNAMICS VIEWPOINT	43
3.6	ALGORITHM VIEWPOINT	43
3.6.1	Activity Diagram	43
4	DEVELOPMENT AND TOOLS.....	45
4.1	INTRODUCTION	45
4.2	DEVELOPMENT PLAN	45
4.3	DEVELOPMENT TOOLS	46
4.4	CONCLUSION AND FUTURE WORK/EXTENSIONS	46
5	QUALITY ASSURANCE	48
5.1	INTRODUCTION	48
5.2	TRACEABILITY MATRIX	48
5.3	TEST PLAN.....	50
6	USER MANUAL.....	59

6.1	INTRODUCTION	59
6.2	HARDWARE/SOFTWARE REQUIREMENTS FOR THE SYSTEM	59
6.3	INSTALLATION GUIDE FOR APPLICATION	60
6.4	OPERATING MANUAL.....	64
6.4.1	Creating an Account	65
6.4.2	Logging In	66
6.4.3	Viewing Bins on Map.....	67
6.4.4	Planning an Optimized Route.....	68
6.4.5	Marking Bins as Collected	69
6.4.6	Managing Profile	70
6.4.7	Admin Panel Operations (Web)	71

LIST OF FIGURES

Figure 2.1: System Design.....	5
Figure 2.2: Use Case Diagram Of System.....	11
Figure 2.3: Use Case Diagram for Login.....	12
Figure 2.4: Use Case Diagram for Manage Users	12
Figure 2.5: Use Case Diagram for Manage Bins	13
Figure 2.6: Use Case Diagram for View Dashboard	14
Figure 2.7: Use Case Diagram for Generate Reports	15
Figure 2.8: : Use Case Diagram for Worker Register.....	15
Figure 2.9: Use Case Diagram for View Bin Status.....	16
Figure 2.10: Use Case Diagram for view Optimized Route	17
Figure 2.11: Use Case Diagram for Receive Alerts.....	17
Figure 2.12: Use Case Diagram for Mark Bin As Collected.....	18
Figure 2.13: Use Case Diagram for Report Bin Anomaly.....	19
Figure 2.14 Login Screen	20
Figure 2.15: Splash Screen	20
Figure 2.16: Map Page.....	21
Figure 2.17: Route Page	21
Figure 2.18: Profile Page.....	22
Figure 2.19: Edit Name	22
Figure 2.20: Confirmation Messages.....	23
Figure 2.21: Error Messages.....	23
Figure 2.22: Validation Guide	23
Figure 3.1: Package Diagram of the System	28
Figure 3.2: Deployment Diagram of the System	29
Figure 3.3: Class Diagram of the System	30
Figure 3.4: Sequence Diagram for Manage Users.....	31
Figure 3.5: Sequence Diagram for Manage Bins.....	32
Figure 3.6: Sequence Diagram for View Dashboard.....	33
Figure 3.7: Sequence Diagram for Generate Reports	34
Figure 3.8: Sequence Diagram for Admin Login	35
Figure 3.9: Sequence Diagram for Worker Login	36
Figure 3.10: Sequence Diagram for Register	37
Figure 3.11: Sequence Diagram for View Bin Status.....	38
Figure 3.12: Sequence Diagram for Plan Optimized Route	39
Figure 3.13: Sequence Diagram for Receive Alerts	40
Figure 3.14: Sequence Diagram for Mark Bin as Collected.....	41
Figure 3.15: Sequence Diagram for Report Bin Anomaly	42
Figure 3.16: State Machine Diagram of The System.....	43
Figure 3.17: Activity Diagram of The System	44
Figure 4.1: Gantt Chart Of Development Plan	46
Figure 6.1: Creating Account	65
Figure 6.2: Logging in Admin.....	66
Figure 6.3: Logging in driver	66
Figure 6.4: Viewing Bins on Map	67
Figure 6.5: Routing.....	68
Figure 6.6: Marking bin Complete by Driver.....	69
Figure 6.7: Updating Profile.....	70
Figure 6.8: Admin Portal.....	71
Figure 6.9: Manage Bins	71
Figure 6.10: Manage Users.....	72
Figure 6.11: Collection Logs	72

LIST OF TABLES

Table 2.1: Functional Requirements	8
Table 2.2: For Login	12
Table 2.3: For Manage Users.....	13
Table 2.4: For Manage Bins	13
Table 2.5: For View Dashboard.....	14
Table 2.6: For Generate Reports.....	15
Table 2.7: For Register	16
Table 2.8: For View Bin Status	16
Table 2.9: For View Optimized Route	17
Table 2.10: For Receive Alerts	18
Table 2.11: For Mark Bin As Collected	18
Table 2.12: For Reporting Bin Anomalies	19
Table 5.1: Traceability Matrix (TC-001 - TC-013).....	48
Table 5.2: Traceability Matrix (TC-014 - TC-025).....	49
Table 5.3: Test Case TC-001 - User Registration.....	50
Table 5.4: Test Case TC-002 - User Login.....	50
Table 5.5: Test Case TC-003 - Profile Management.....	51
Table 5.6: Test Case TC-004 - Password Reset	51
Table 5.7: Test Case TC-005 - Session Management.....	51
Table 5.8: Test Case TC-006 - Bin Simulation	52
Table 5.9: Test Case TC-007 - AI Fill Prediction	52
Table 5.10: Test Case TC-008 - Bin Data Storage	52
Table 5.11: Test Case TC-009 - Real-time Bin Status	53
Table 5.12: Test Case TC-010 - Interactive Map Display.....	53
Table 5.13: Test Case TC-011 - Route Planning.....	53
Table 5.14: Test Case TC-012 - Route Optimization	54
Table 5.15: Test Case TC-013 - Route Visualization.....	54
Table 5.16: Test Case TC-014 - Mark as Collected	54
Table 5.17: Test Case TC-015 - Report Anomaly	55
Table 5.18: Test Case TC-016 - Real-time Route Updates	55
Table 5.19: Test Case TC-017 - Push Notifications	55
Table 5.20: Test Case TC-018 - Admin Dashboard	56
Table 5.21: Test Case TC-019 - Anomaly Management.....	56
Table 5.22: Test Case TC-020 - Performance Reports.....	56
Table 5.23: Test Case TC-021 - User Management	57
Table 5.24: Test Case TC-022 - Bin Management	57
Table 5.25: Test Case TC-023 - Zone Assignment	57
Table 5.26: Test Case TC-024 - Time Slot Scheduling.....	58
Table 5.27: Test Case TC-025 - Historical Data Access	58

1 INTRODUCTION

This chapter introduces Clean Core, its purpose, background, objectives, and significance. It provides the context and motivation behind developing an AI-driven waste monitoring and optimization system that integrates computer vision-based fill prediction with intelligent route optimization for waste collection drivers.

1.1 System Introduction

Clean Core is a mobile and web-based suite based on Flutter that can greatly improve the efficiency of municipal waste management. The system is based on a mobile application that replaces the static route planning with dynamic, demand-based logistics on a driver facing platform. This is accomplished with a backend Machine Learning (ML) model to show the actual fill level and type of waste of sanitation bin at the applicable real-time through simulated data. According to such predictions, the system uses a sophisticated Vehicle Routing Problem (VRP) algorithm by dynamically determining the most optimal and efficient gathering path to be undertaken by the driver. This will enable the sanitation teams to work on high-priority bins, which will result in the reduction of operational costs, carbon emissions, and better service delivery. There is also real time reporting and a full Admin Panel where the entire system can be centrally monitored, and users managed background of the System.

1.2 Background of the System

The conventional system of municipal waste collection is based on fixed and pre-defined routes, without considering the fill level of each bin. This causes a high level of inefficiencies since trucks will go round with half empty bins which will waste their fuel, cause unnecessary mileage and high operating expenses. Moreover, the use of scheduled pickups might lead to surplus in the bins of the busy or unforeseeable places prior to the next scheduled pickup, which will lead to sanitation and health problems to the population. Clean Core eliminates these deficiencies as it takes the waste management to the next stage of smart and data logistics. The context of the project is the necessity to find the solution based

on the predictive analytics and real-time optimization to convert the old process into the smart demand-driven one so that resources could be deployed only at the time and place where they are needed most.

1.3 Objectives of the System

The primary objectives of Clean Core are to introduce efficiency, intelligence, and transparency into the waste management process. These goals will be achieved through the following specific objectives:

- **Optimize Collection Routes:**

To implement an advanced Vehicle Routing Problem (VRP) algorithm that dynamically generates the shortest and most efficient collection route for drivers, prioritizing bins based on their predicted fill levels to minimize travel time and fuel consumption.

- **Enable Demand-Based Collection:**

To develop a backend Machine Learning model capable of accurately predicting the fill level and waste type of sanitation bins, allowing the system to shift from fixed schedules to a data-driven, demand-based collection strategy that is more accurate.

- **Provide Real-Time Operational Visibility:**

To equip drivers with a mobile application interface that offers real-time map-based visualization of all bin statuses, current location, and the dynamic optimized route, thereby improving decision-making operations.

- **Enhance Service Efficiency and Reliability:**

To reduce instances of overflowing bins and collection delays by ensuring high-urgency bins are serviced promptly, consequently improving urban sanitation and public satisfaction.

- **Centralize Management and Reporting:**

To create a secure Admin Panel for supervisory personnel to manage users, monitor operational status, track driver performance, and access analytics related to collection statistics and system utilization.

- **Centralize Management and Reporting:**

To establish a comprehensive backend mechanism for simulating bin location data, fill level sensor input, and image data to effectively train the ML mode and demonstrate the full functionality of the system in a controlled environment.

1.4 Significance of the System

The development of Clean Core holds significant value by modernizing municipal services and addressing critical challenges in urban resource management. For Waste Management Authorities, the system provides a clear pathway to substantial cost savings by optimizing truck routes and reducing unnecessary operational expenses, maximizing the utility of every collection run. From an environmental perspective, reduced travel distances directly translate into lower fuel consumption and a decreased carbon footprint, contributing to city-wide sustainability goals. For Sanitation Drivers, the app simplifies their workday by providing clear, optimal guidance and reducing the ambiguity of manual route planning, thereby increasing their efficiency and job satisfaction. Most importantly, the system's ability to ensure proactive collection specifically targeting bins predicted to be full results in a cleaner, more hygienic urban environment, enhancing the quality of life for all citizens and proactively mitigating public health risks associated with overflowing waste containers.

2 REQUIREMENT SPECIFICATIONS

This chapter provides a detailed overview of the Clean Core system, covering its product perspective, scope, and core functionality. It defines the system's four major functional areas: User Management, Intelligent Data Processing, Route Optimization, and System Administration. The chapter also identifies the primary users Workers and Administrators along with their roles, privileges, and usage characteristics. Finally, it specifies the operating environment requirements for both the mobile application and the administrative web panel, including hardware, software, and connectivity prerequisites essential for system deployment.

2.1 Product Scope

Clean Core is a Flutter-based mobile application focused on AI-driven waste monitoring and route optimization for sanitation drivers. It includes user authentication, real-time bin visualization on maps, YOLO-based fill level prediction, optimized route planning using VRP algorithm, collection workflow management, anomaly reporting, and a React-based admin panel for user management, bin configuration, and report generation.

2.2 Product Description

2.2.1 Product Perspective

Clean Core being developed as a new, self-contained product aimed at modernizing and replacing outdated, paper-based, or fixed-schedule route management systems currently employed by sanitation departments. It is not part of a pre-existing product family but is designed to function as the central intelligence layer for optimizing municipal solid waste logistics. The system integrates several key technological components mobile development, predictive analytics, and spatial optimization into a unified platform. Its primary interfaces are the Driver Mobile Application and the Administrator Web Panel.

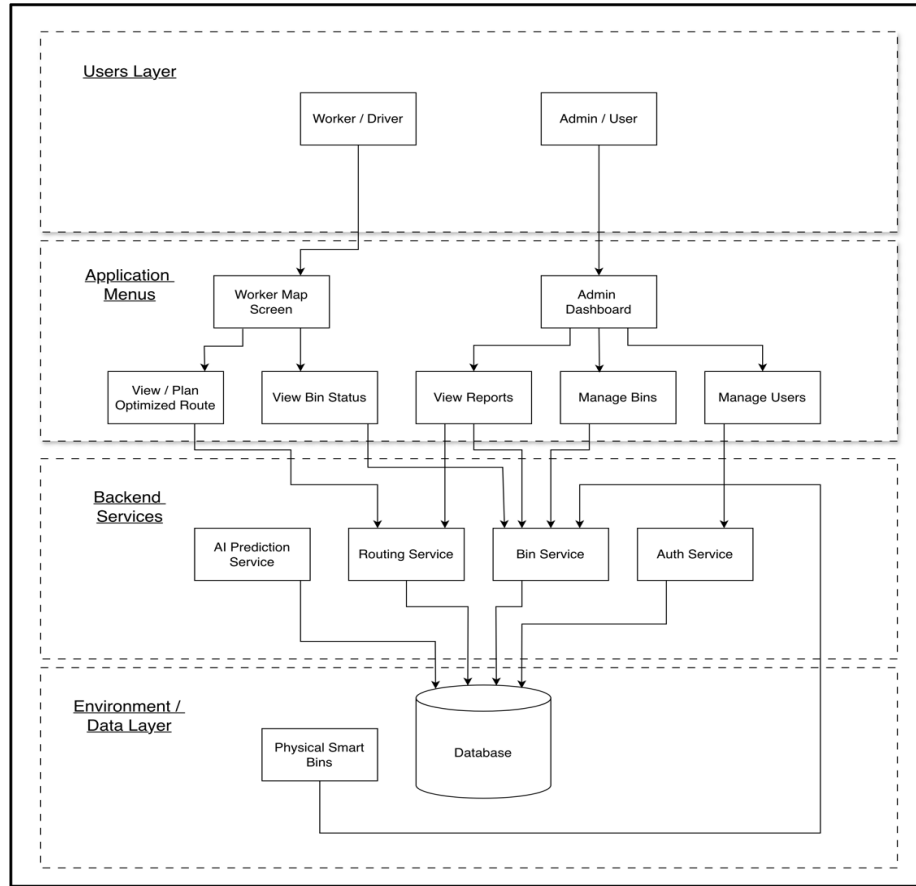


Figure 2.1: System Design

2.2.2 Product Functionality

Clean Core is designed to perform and facilitate four major categories of functions, covering user access, data intelligence, operational logistics, and system management.

- **User Management and Access:**
 - Secure login and authentication for both Sanitation Drivers (via mobile app) and Administrators (via web panel).
 - User role and authorization management to control access to system features based on user type.

- **Intelligent Data Processing (Backend):**
 - Simulated Bin Status Generation to provide continuous, dynamic location and fill-level data for all bins.
 - Machine Learning (ML) Prediction of bin fill level and dominant waste type.
 - Data Persistence to securely store all bin data, user profiles, and operational logs.
- **Route Optimization and Execution (Driver App):**
 - Interactive Map Display showing all bins and their predicted status (fill level).
 - Route Planning Algorithm Execution (VRP Solver) to dynamically calculate the most optimal collection sequence for a driver's selected area.
 - Route Guidance and Navigation display of the calculated optimal path on the map.
 - Collection Workflow Management allowing drivers to mark bins as 'Collected' or 'Report Anomaly.'
- **System Administration and Reporting (Admin Panel):**
 - Operational Monitoring to view the real-time status of all trucks, drivers, and collection progress.
 - Performance Reporting to generate statistics on collection efficiency, mileage, and anomaly rates.
 - User and Bin Data Management including the ability to add, modify, or deactivate bins and users.

2.2.3 Users and Characteristics

- **Worker:**
 - **Frequency of Use:** High (Multiple times daily during shifts).

- **Privilege Level:** Low. Restricted to their assigned routes, viewing live bin status, and updating collection logs.
- **Environment:** Mobile, Outdoor, often in varying light and weather conditions, necessitating a highly readable and responsive mobile interface.
- **Educational Level:** Varies; the interface must be accessible regardless of educational background, relying heavily on visual aids and clear labeling.
- **Admin:**
 - **Frequency of Use:** Moderate (Daily checks and weekly reporting).
 - **Privilege Level:** High. Has read/write access to user data, bin data configuration, and the ability to generate all system reports.
 - **Technical Focus:** Primarily concerned with data accuracy, operational efficiency metrics (e.g., cost savings, bins collected), and user security.

2.2.4 Operating Environment

- **Mobile Client (Sanitation Driver)**

The driver application, built using Flutter, must be accessible and robust in a field environment.

- **Platform:** Android and iOS.
- **Minimum Operating System:** Android 10 or higher, iOS 14 or higher.
- **Hardware Requirements:**
 - **Processor:** Quad-core 1.5 GHz or better.
 - **RAM:** Minimum 4 GB.
 - **Storage:** At least 200 MB free for app installation and cached data.

- **Connectivity:** Consistent access to 4G/LTE/5G or Wi-Fi for real-time data sync, though the application must handle temporary disconnections gracefully by caching data locally.
- **Peripherals:** Functional GPS hardware for location tracking and navigation.
- **Web Client (Administrator)**

The administrator panel will be a web application accessed through standard web browsers.

 - **Browser Requirements:** The latest stable versions of Google Chrome, Mozilla Firefox, Microsoft Edge, or Apple Safari.
 - **Hardware Requirements:** Standard desktop or laptop computer.
 - **Minimum Display Resolution:** 1280 x 1024 pixels.

2.3 Specific Requirements

2.3.1 Functional Requirements

Table 2.1: Functional Requirements

Requirement Number	Requirement	Description
SFR-001	User Registration	Users must be able to create accounts using email via Firebase Authentication.
SFR-002	User Login	Users must be able to log in with credentials or single sign-in methods.
SFR-003	Profile Management	Users should be able to update profiles including name, phone number, and profile picture.
SFR-004	Password Reset	Users must be able to reset their passwords via a secure email verification process.

SFR-005	Session Management	The system must maintain a secure session until user logs out or session times out due to inactivity.
SFR-006	Bin Simulation	The system must generate and update fill levels for all bins at regular intervals to simulate real-world usage.
SFR-007	AI Fill Prediction	The YOLO-based ML model must predict bin fill level (Empty, Low, Medium, Full) from image data.
SFR-008	Bin Data Storage	The system must store all bin location data, fill levels, and historical collection records in the database.
SFR-009	Real-time Bin Status	The system must ensure bin status data retrieved by the mobile app is no older than 15 minutes.
SFR-010	Interactive Map Display	The mobile app must display an interactive map showing driver's location and markers for all assigned bins.
SFR-011	Route Planning	The system must allow drivers to initiate route planning using bins predicted to be Full or Medium as stops.
SFR-012	Route Optimization	The VRP algorithm must calculate the optimal route balancing fill level urgency and travel distance.
SFR-013	Route Visualization	The mobile app must display the calculated optimal route as a clear path overlaid on the map.
SFR-014	Mark as Collected	Drivers must be able to mark bins as collected, recording timestamp and removing stop from active route.
SFR-015	Report Anomaly	Drivers must be able to report bin anomalies with pre-defined reasons (e.g., Blocked, Damaged).
SFR-016	Real-time Route Updates	The mobile app must update route display and remaining stops in real-time as bins are collected.
SFR-017	Push Notifications	Workers must receive real-time notifications for route assignments and critical alerts.

SFR-018	Admin Dashboard	Admin must be able to view real-time operational status of all active drivers and collection progress.
SFR-019	Anomaly Management	Admin must be able to view and filter all unresolved anomalies including location, timestamp, and reporter.
SFR-020	Performance Reports	Admin must be able to generate reports on mileage saved, collection time, and bins collected over date ranges.
SFR-021	User Management	Admin must be able to create, modify, and deactivate user accounts with assigned roles.
SFR-022	Bin Management	Admin must be able to add, edit, and delete bins with location and zone information.
SFR-023	Zone Assignment	Admin must be able to define and assign specific collection zones to different driver accounts.
SFR-024	Time Slot Scheduling	Admin must be able to set and manage collection time slots for workers.
SFR-025	Historical Data Access	Admin must be able to view and download historical collection data for analysis and ML validation.

2.3.2 Behaviour Requirements

- Use Case Diagram for Clean Core

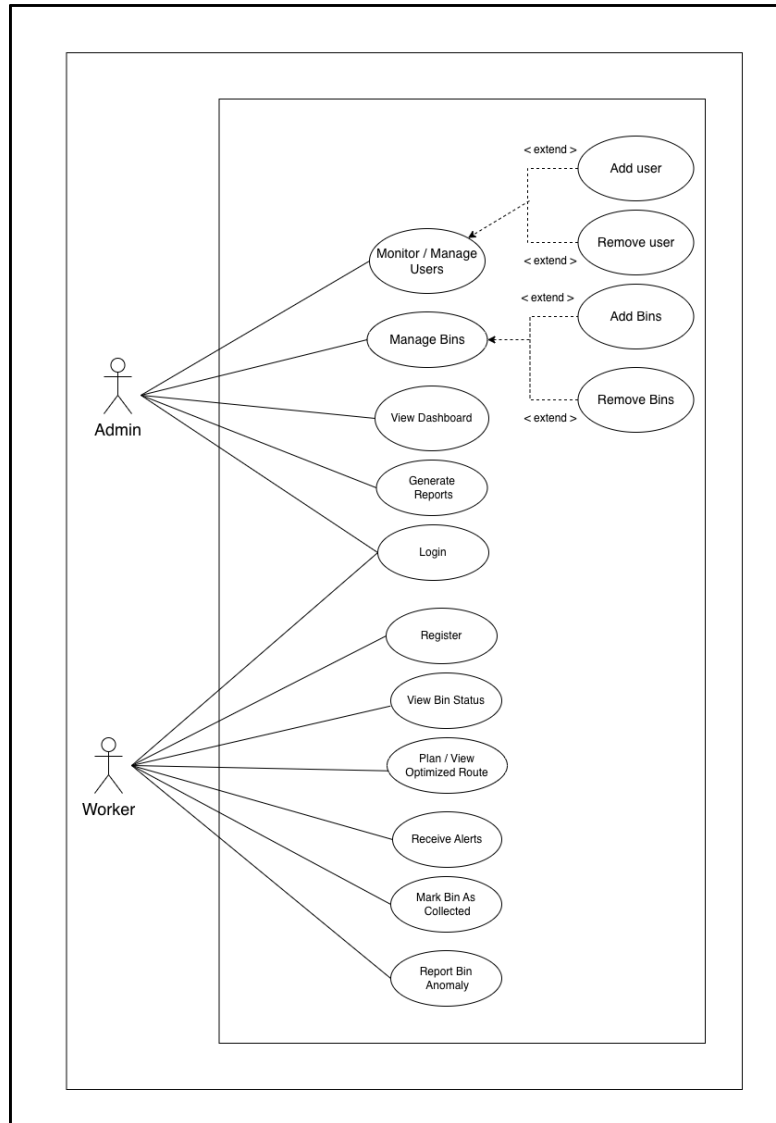


Figure 2.2: Use Case Diagram Of System

- **Use Case # 1: Login**

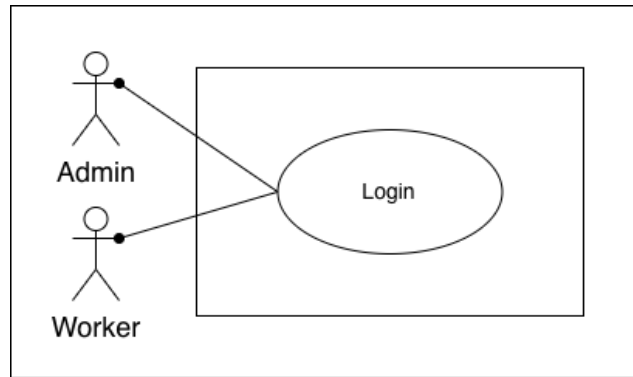


Figure 2.3: Use Case Diagram for Login

Table 2.2: For Login

Field	Details
Actors	Admin, Worker
Description	A user logs into the system to access its features
Pre - condition	The user must have a registered account
Steps	1. User enters credentials 2. System verifies credentials 3. System grants access and redirects to dashboard
Post Condition	User is authenticated and gains access.

- **Use Case # 2: Manage Users**

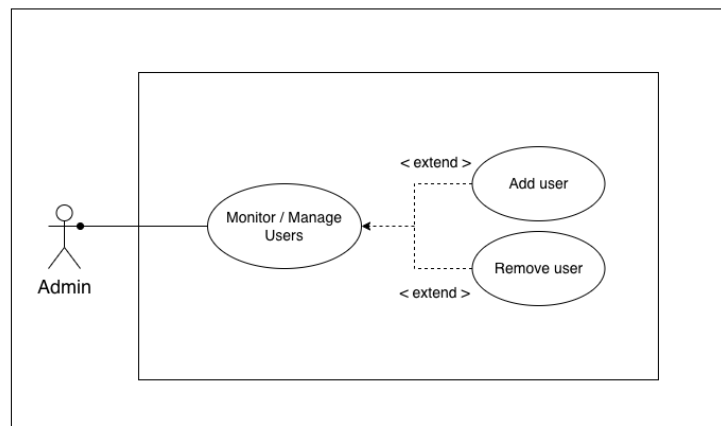


Figure 2.4: Use Case Diagram for Manage Users

Table 2.3: For Manage Users

Field	Details
Actors	Admin
Description	Admin manages the accounts of workers
Pre - condition	Admin must be logged into the system.
Steps	1. Admin logs into system 2. Admin selects manage user's option 3. Admin chooses to add or remove users 4. System facilitates chosen option
Post Condition	Admin can create or delete worker accounts
Relationships	Extends: Add user, remove user

- **Use Case # 3: Manage Bins**

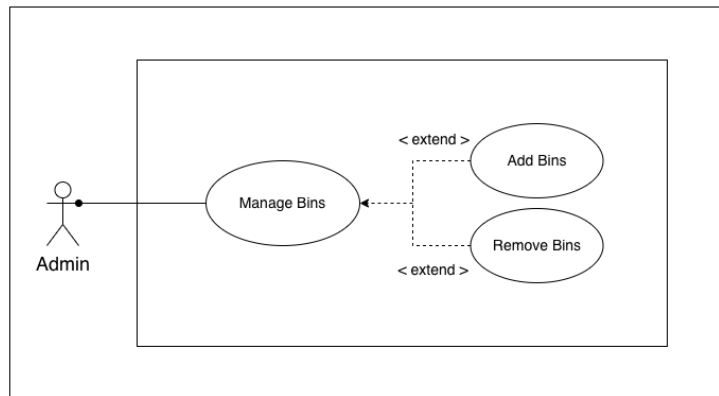


Figure 2.5: Use Case Diagram for Manage Bins

Table 2.4: For Manage Bins

Field	Details
Actors	Admin
Description	Admin manages the number of bins and their description
Pre - condition	Admin must be logged into the system.
Steps	1. Admin logs into system 2. Admin selects manage bins option 3. Admin chooses to add or remove users

	4. System facilitates chosen option
Post Condition	Admin can create or delete bins
Relationships	Extends: Add bin, remove bin

- **Use Case # 4: View Dashboard**

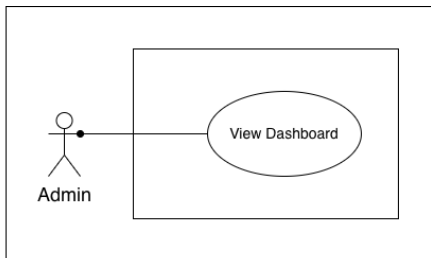


Figure 2.6: Use Case Diagram for View Dashboard

Table 2.5: For View Dashboard

Field	Details
Actors	Admin
Description	Admin monitors collection routes, routines, and bin statuses
Pre - condition	Admin must be logged into the system
Steps	1. Admin logs into system 2. Admin selects view dashboard option 3. System displays dashboard 4. Admin reviews the data
Post Condition	Admin receives dashboard of routes, routines, and bin statuses

- **Use Case # 5: Generate Reports**

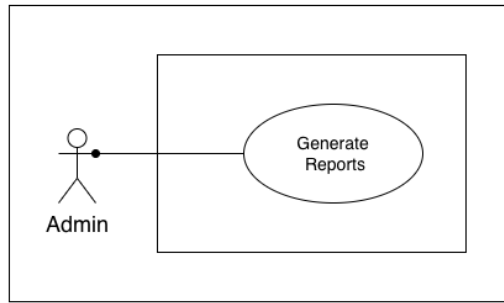


Figure 2.7: Use Case Diagram for Generate Reports

Table 2.6: For Generate Reports

Field	Details
Actors	Admin
Description	Admin monitors worker activities to ensure proper usage
Pre - condition	Admin must be logged into system
Steps	1. Admin logs into system. 2. Admin selects get report option 3. System displays user activity data with a report 4. Admin reviews the data
Post Condition	Admin receives report of user activity

- **Use Case # 6: Register**

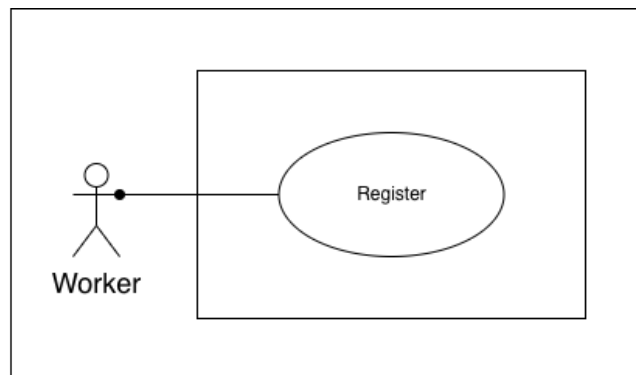


Figure 2.8: : Use Case Diagram for Worker Register

Table 2.7: For Register

Field	Details
Actors	Worker
Description	Worker creates an account to access the system
Pre - condition	Worker must not already have an account
Steps	1. Worker accesses registration page 2. Worker provides required details 3. System validates input 4. System creates account
Post Condition	Worker is registered and can log in

- **Use Case # 7: View Bin Status**

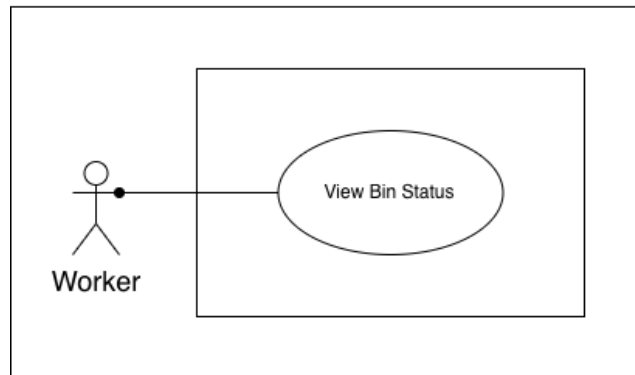


Figure 2.9: Use Case Diagram for View Bin Status

Table 2.8: For View Bin Status

Field	Details
Actors	Worker
Description	Worker can check bins fill level and waste type
Pre - condition	Worker must be logged into system
Steps	1. Worker logs into system 2. Worker clicks on a particular bin icon on map 3. Worker sees bin status
Post Condition	Description of bin fill level, waste type is displayed to the worker

- **Use Case # 8: View Optimized Route**

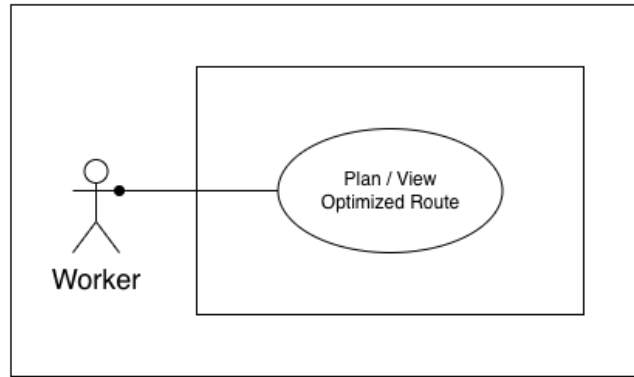


Figure 2.10: Use Case Diagram for view Optimized Route

Table 2.9: For View Optimized Route

Field	Details
Actors	Worker
Description	Worker can see optimized collection route
Pre - condition	Worker must be logged into system
Steps	1. Worker selects plan route option 2. System plans optimal route and displays it on map for worker to follow
Post Condition	Optimal planned route is displayed by system

- **Use Case # 9: Receive Alerts**

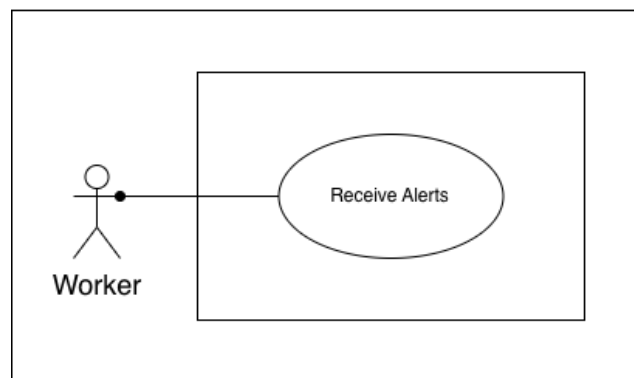


Figure 2.11: Use Case Diagram for Receive Alerts

Table 2.10: For Receive Alerts

Field	Details
Actors	Worker
Description	Worker receives alert when it's time for bin collection
Pre - condition	Worker has a registered account
Steps	1. Worker registers themselves 2. Admin sets collection times 3. Worker receives alerts based on time
Post Condition	Notification is marked as read

- **Use Case # 10: Mark Bin as Collected**

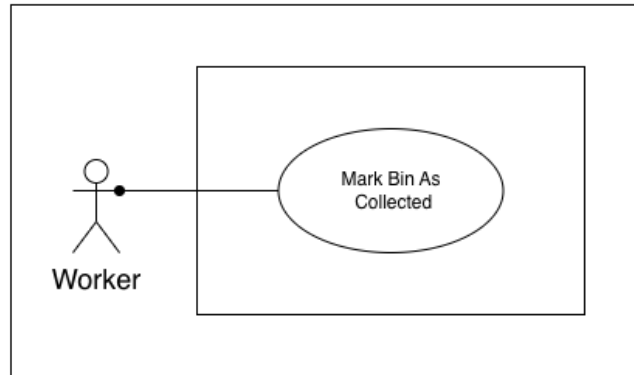


Figure 2.12: Use Case Diagram for Mark Bin As Collected

Table 2.11: For Mark Bin As Collected

Field	Details
Actors	Worker
Description	After collection, worker marks bin as collected
Pre - condition	Worker follows planned route to reach
Steps	1. Worker plans route 2. Worker follows route to reach bin 3. Worker marks it as collected after collection
Post Condition	Bin is marked as collected and bin fill level is back to empty

- **Use Case # 11: Report Bin Anomaly**

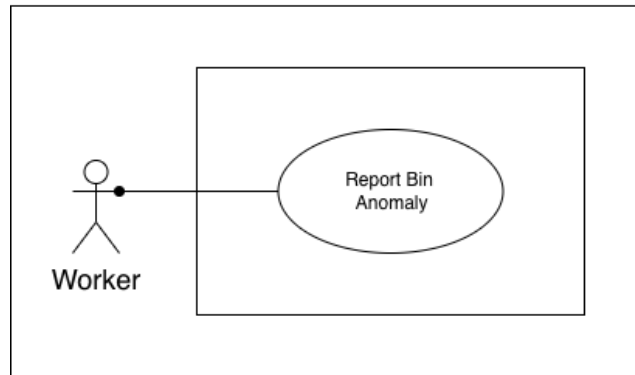


Figure 2.13: Use Case Diagram for Report Bin Anomaly

Table 2.12: For Reporting Bin Anomalies

Field	Details
Actors	Worker
Description	Worker can report any anomaly regarding the bin
Pre - condition	Worker has access to bin data
Steps	1. Worker logs into system 2. Worker spots anomaly in a bin 3. Worker fills anomaly report form 4. Worker presses report anomaly button
Post Condition	Anomaly report is forwarded to admin

2.3.3 External Interface Requirements

User Interface

The Clean Core mobile application provides a clean, intuitive interface optimized for outdoor use. The main navigation uses a bottom tab bar with icons for Home, Map, Route, and Profile. The map screen displays color-coded bin markers (Green=Empty, Yellow=Low, Orange=Medium, Red=Full) on an interactive Google Maps view. Route visualization shows the optimized path with numbered waypoints. All touch targets are sized appropriately for gloved hands and varying light conditions.

User Interfaces

Splash Screen:

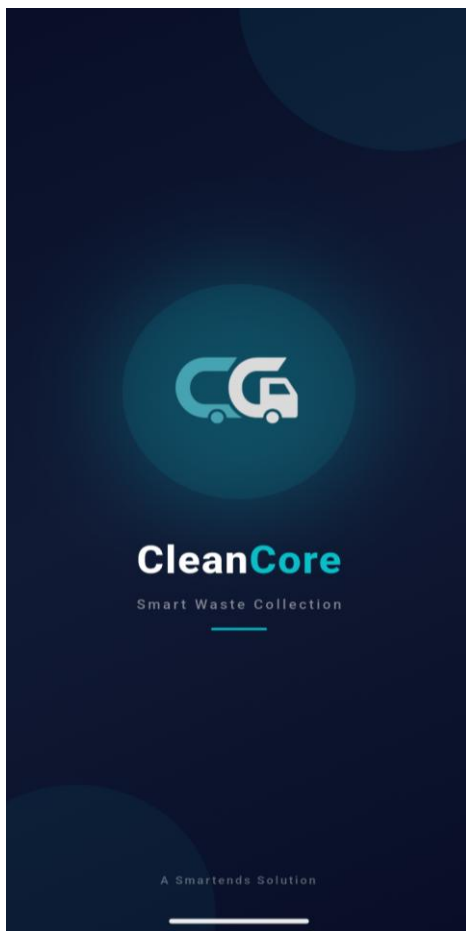


Figure 2.15: Splash Screen

User Authentication Screen:

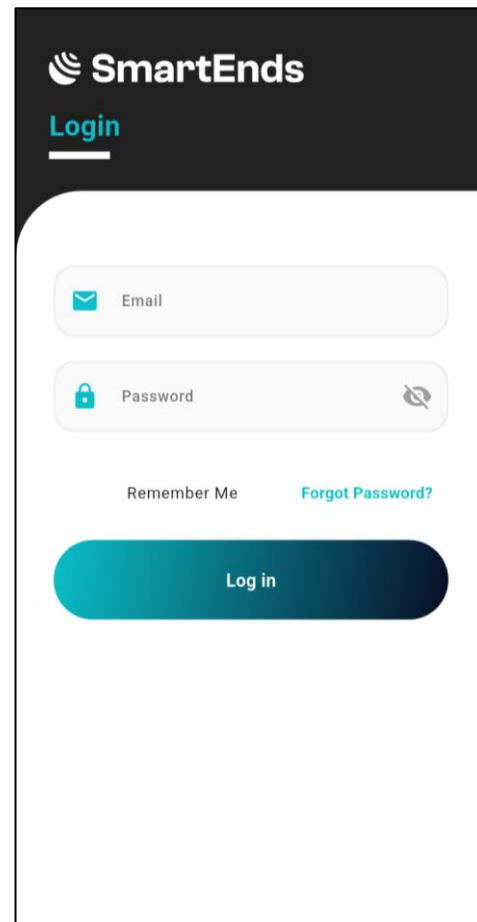


Figure 2.14 Login Screen

- **Features:** Secure user authentication via Email and Google Sign-in.
- **Standard Buttons:** "Login", "Sig in with Google", "Forgot Password".

- **Main Screens:**

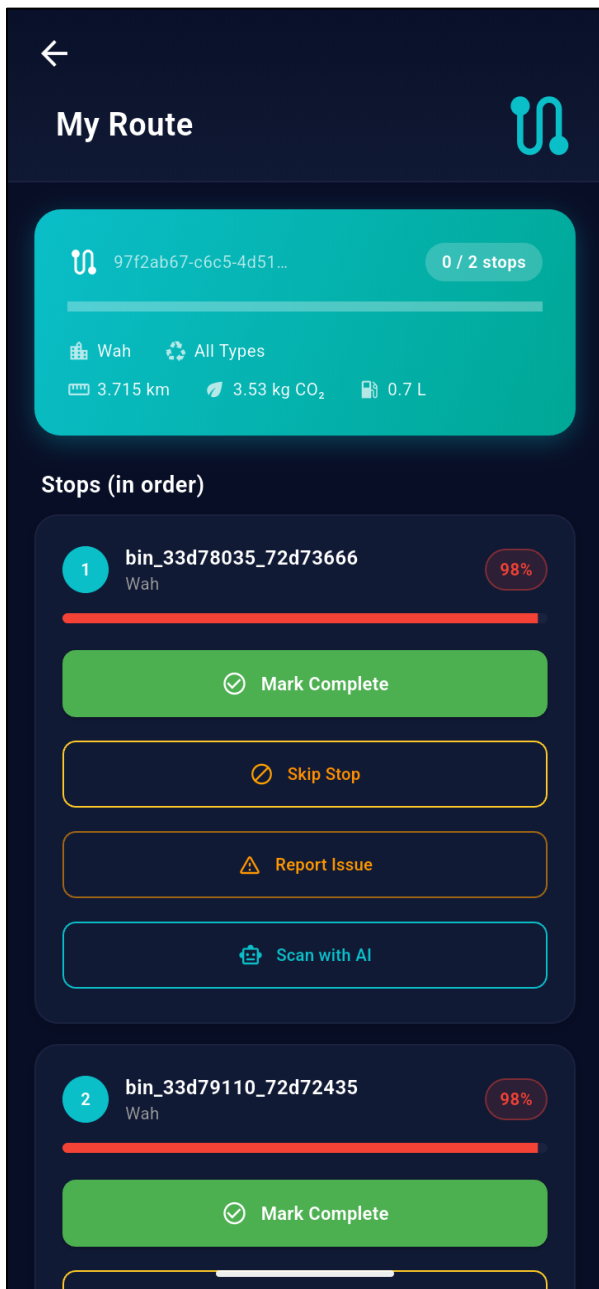


Figure 2.17: Route Page

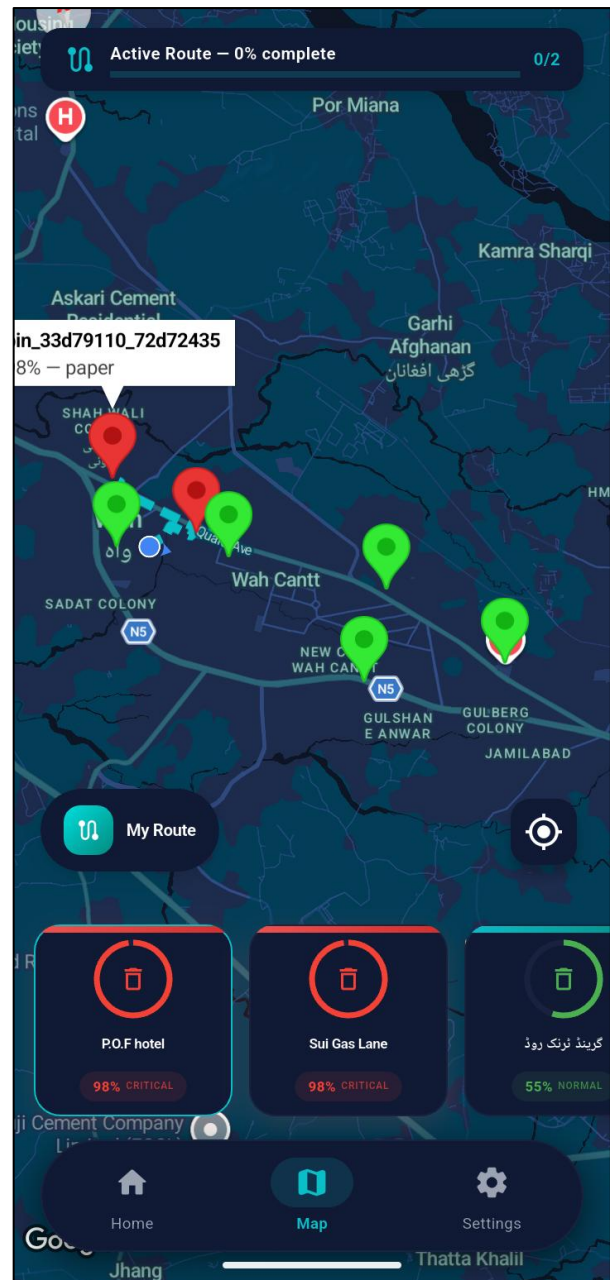


Figure 2.16: Map Page

- **Features:** Quick overview of today's route, important alerts, and notifications

- **Personal Profile Page:**

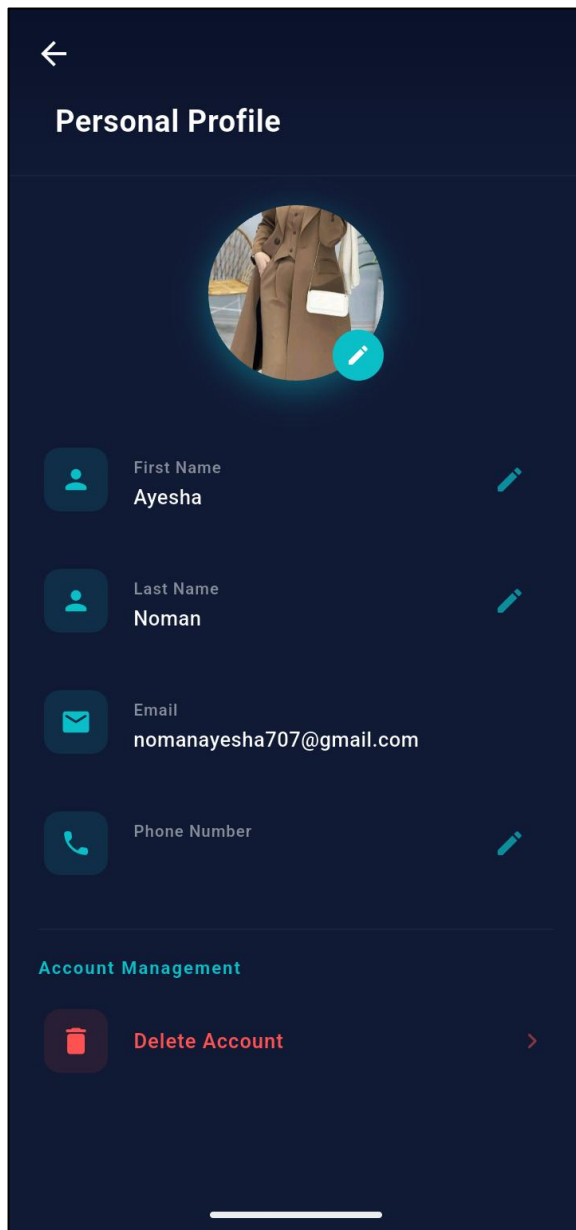


Figure 2.18: Profile Page

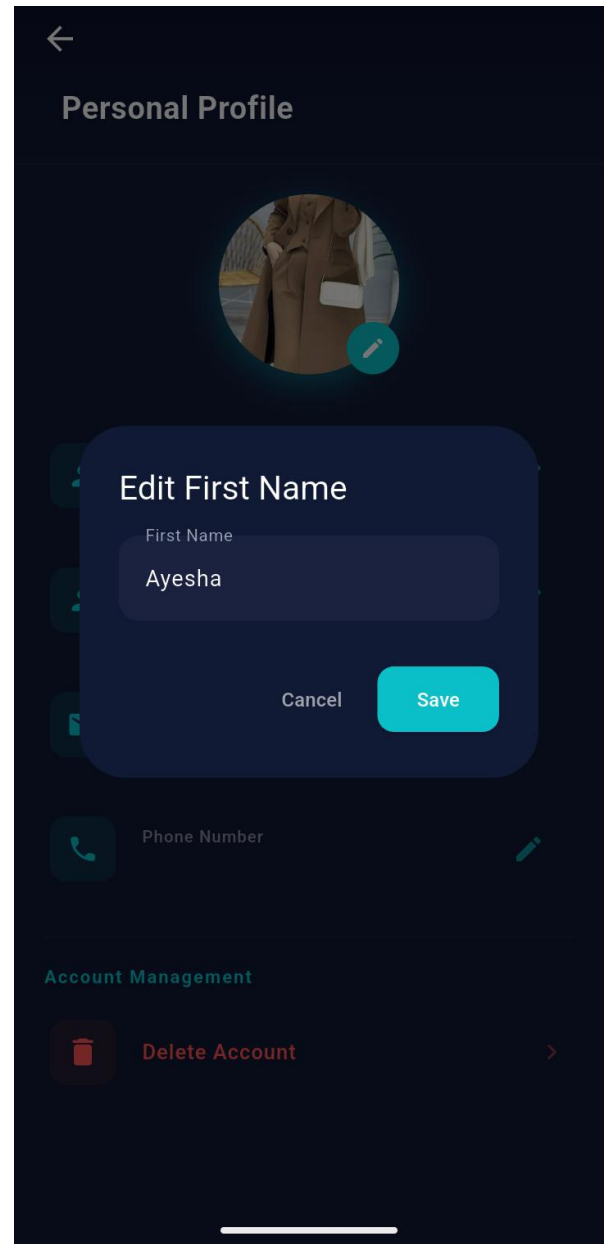


Figure 2.19: Edit Name

- **Features:** Edit personal details, delete account or sign-out.
- **Standard Buttons:** Edit first name, edit last name, edit phone number, Delete account, Sign-out

- **Error Message Display Standards:**

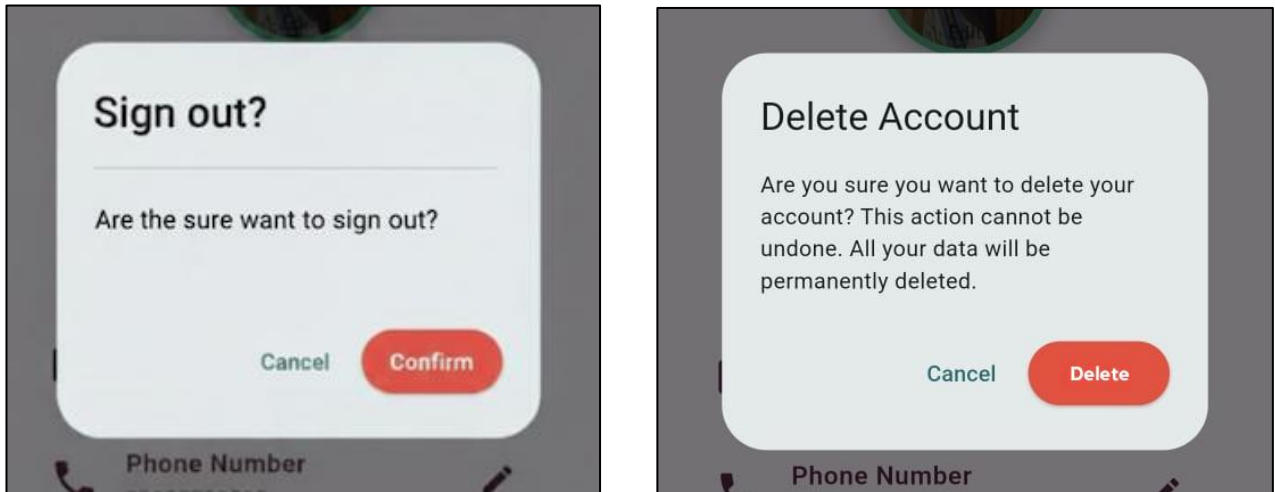


Figure 2.20: Confirmation Messages

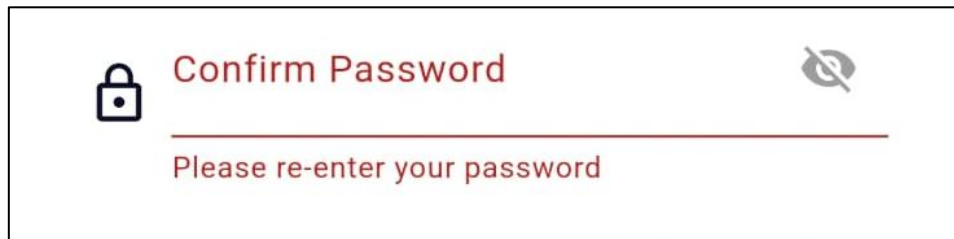


Figure 2.21: Error Messages

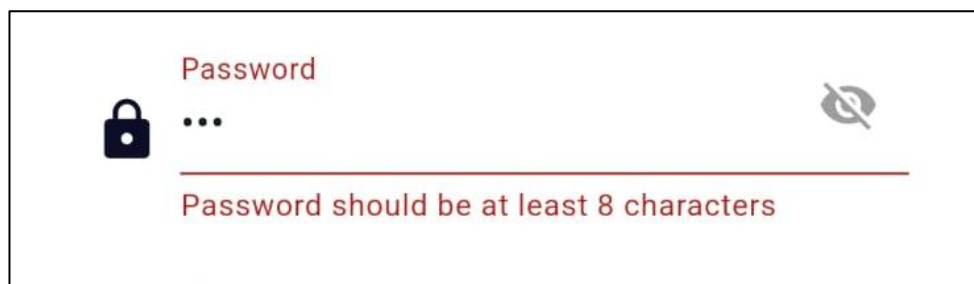


Figure 2.22: Validation Guide

Hardware Interfaces:

The mobile application interfaces with the device's GPS hardware for real-time location tracking, camera for potential future bin image capture, and network hardware (cellular/Wi-Fi) for data synchronization. The admin panel requires standard web browser capabilities and network connectivity.

Software Interfaces:

- **Firestore Authentication:** User authentication and session management
- **Firestore:** Real-time NoSQL database for bin and user data
- **Cloud Messaging:** Push notifications
- **Google Maps API:** Map display, geocoding, and directions
- **FastAPI Backend:** Python-based API for YOLO model inference and VRP route optimization

Communication Interfaces:

All communication between clients and backend services uses HTTPS for secure data transmission. The mobile app communicates with Firebase services using Firebase SDK protocols. Real-time data synchronization is achieved through Firestore's WebSocket connections. The FastAPI backend uses REST API endpoints for AI inference and route optimization requests.

2.4 Non-Functional Requirements

2.4.1 Performance Requirements

- **Route Planning Time:**
VRP algorithm must calculate optimal route for up to 50 stops within 5 seconds.
- **Map Startup Time:**
Worker App must initialize map and load all bin markers within 3 seconds.
- **Real-time Synchronization:**
Bin status updates must reflect on Admin Dashboard within 1 second.

- **AI Prediction Latency:**
YOLO model must process image and return prediction within 2 seconds.
- **General Response Time:**
All API interactions should complete within 500 milliseconds for 95% of requests.

2.4.2 Safety and Security Requirements

Safety Requirements:

- System must prevent loss of bin data, routes, and user information with Firebase redundancy.
- If AI or routing services fail, app must show warning and allow manual actions without crashing.
- Outdated bin data must show timestamp and warn user about GPS or network issues immediately.

Security Requirements:

- Firebase Authentication with strong passwords and role-based access (Worker/Admin).
- All data transmitted over HTTPS with no sensitive data stored in plain text.
- Workers only access worker functions; admins access admin-only features.
- Daily database backups with ability to restore in case of accidental deletion.

2.4.3 Software Quality Attributes

- **Availability:**
System shall achieve 97% uptime leveraging Firebase's reliable cloud infrastructure.
- **Maintainability:**
Modular MVVM architecture with documented codebase for easy updates.
- **Portability:**
Flutter codebase can compile for both Android and iOS platforms.

- **Reliability:**
Comprehensive error handling ensures consistent performance under high load.
- **Usability:**
Intuitive UI with clear icons and bottom navigation for ease of use.
- **Scalability:**
System must support 1,000 bins on map and handle 50,000 read/write operations per day.

3 DESIGN SPECIFICATIONS

This chapter presents the comprehensive architectural and detailed design of Clean Core. It begins with the Composite Viewpoint, illustrating the high-level modular structure and integration of the Flutter frontend, Firebase backend, Python AI services, and React admin panel. The Logical Viewpoint details class relationships and core components such as authentication, bin management, route optimization, and admin moderation. The Interaction Viewpoint provides sequence diagrams for all major use cases. The State Dynamics Viewpoint describes state transitions during key operations, while the Algorithmic Viewpoint includes activity diagrams and pseudocode reflecting the implemented behavior of the system.

3.1 Introduction

This chapter provides a multi-perspective view of Clean Core's software architecture and design. Each viewpoint addresses a different aspect of the system: composite structure, logical organization, dynamic interactions, state transitions, and algorithmic behavior. Together, these viewpoints ensure a comprehensive understanding of how the system is designed, how components interact, and how the system responds to user actions and events.

3.2 Composite Viewpoint

The Composite Viewpoint identifies the major design constituents of Clean Core, including composition and modular assembly of systems in terms of subsystems. This section presents Package Diagrams and Deployment Diagrams that illustrate the system's high-level architecture.

3.2.1 Package Diagram:

Workers interact with the Clean Core mobile application (built with Flutter) to perform actions such as user authentication, viewing bin locations and fill levels, planning optimized routes, marking bins as collected, and reporting anomalies. Admins use the React web panel to manage users, bins, zones, and generate reports. The Flutter app communicates with Firebase services (Authentication, Firestore,

Storage, Cloud Messaging) for real-time data synchronization. The Python backend hosts the YOLO AI model for fill level prediction and the VRP algorithm for route optimization.

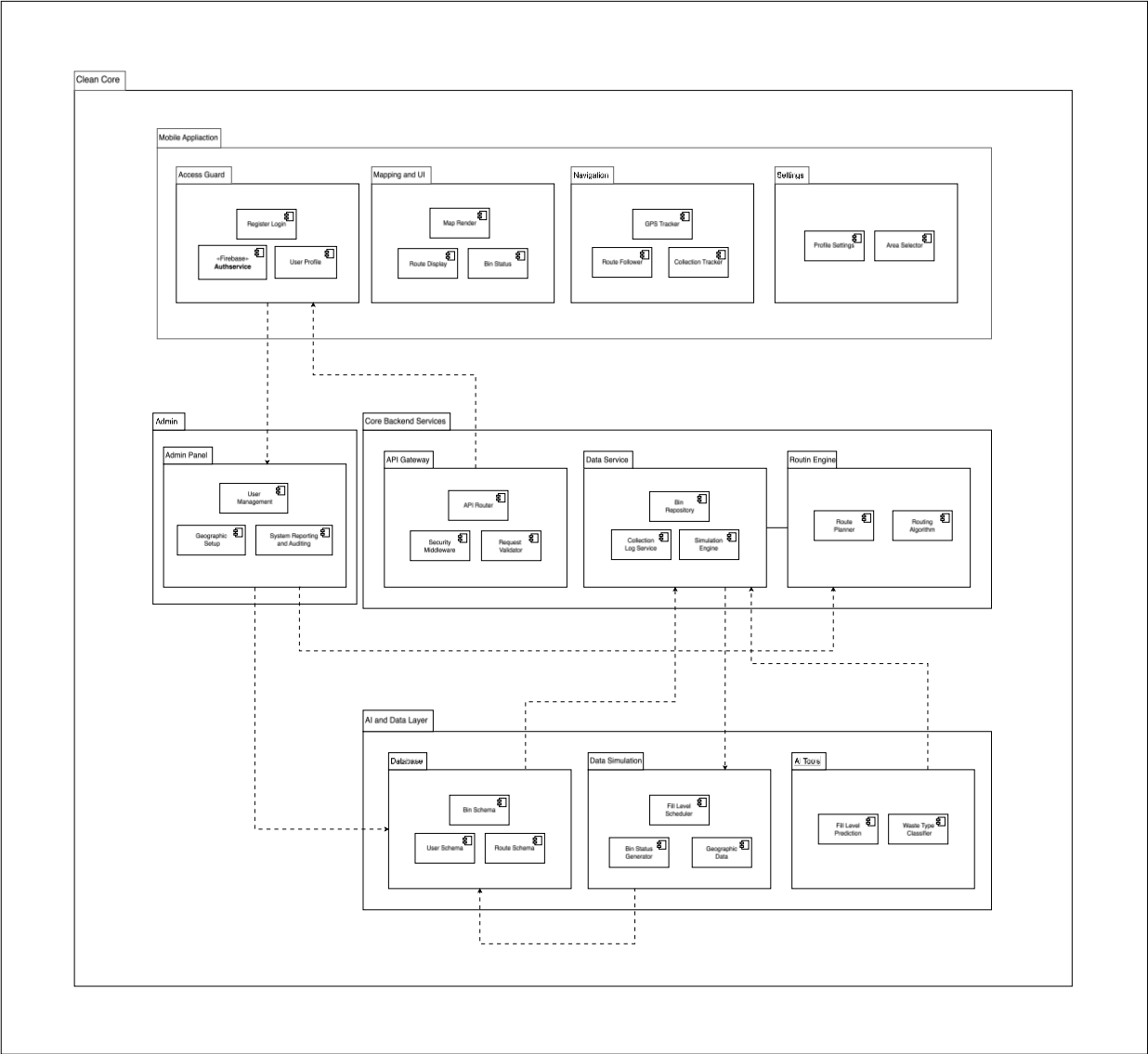


Figure 3.1: Package Diagram of the System

3.2.2 Deployment Diagram (Physical)

The Deployment Diagram shows the physical distribution of the Clean Core system across multiple nodes, including user mobile devices running the Flutter app, the Firebase Cloud (Firestore, Authentication, Storage), and the Python-based backend services for AI prediction and route planning. It illustrates HTTPS communication links between components and depicts real-time syncing of bin updates and route data across cloud services.

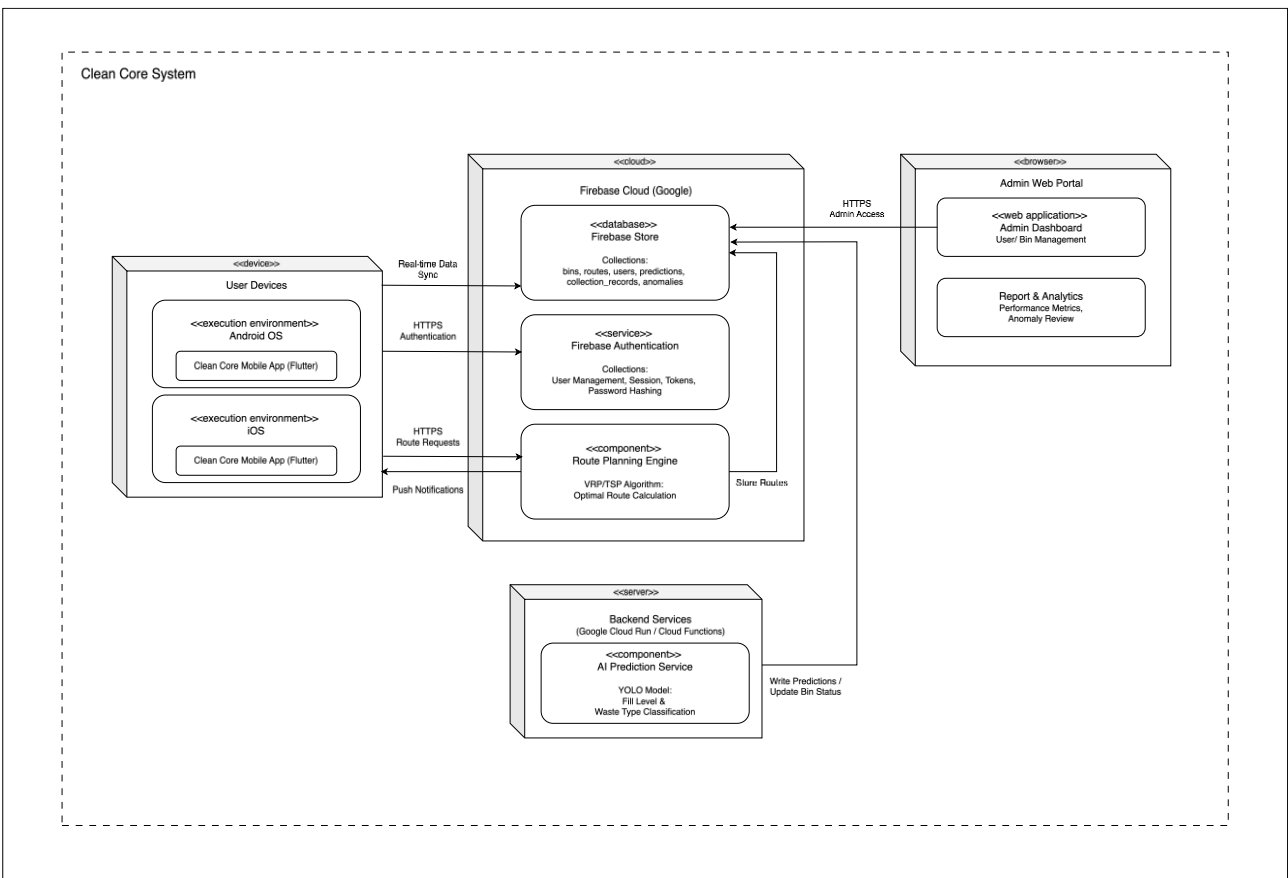


Figure 3.2: Deployment Diagram of the System

3.3 Logical Viewpoint

The Logical Viewpoint elaborates existing and designed types and their implementations as classes and interfaces with their structural static relationships. The Class Diagram depicts main entities (Users, Bins, Routes, Predictions) and their relationships, along with service and controller classes that manage system behavior.

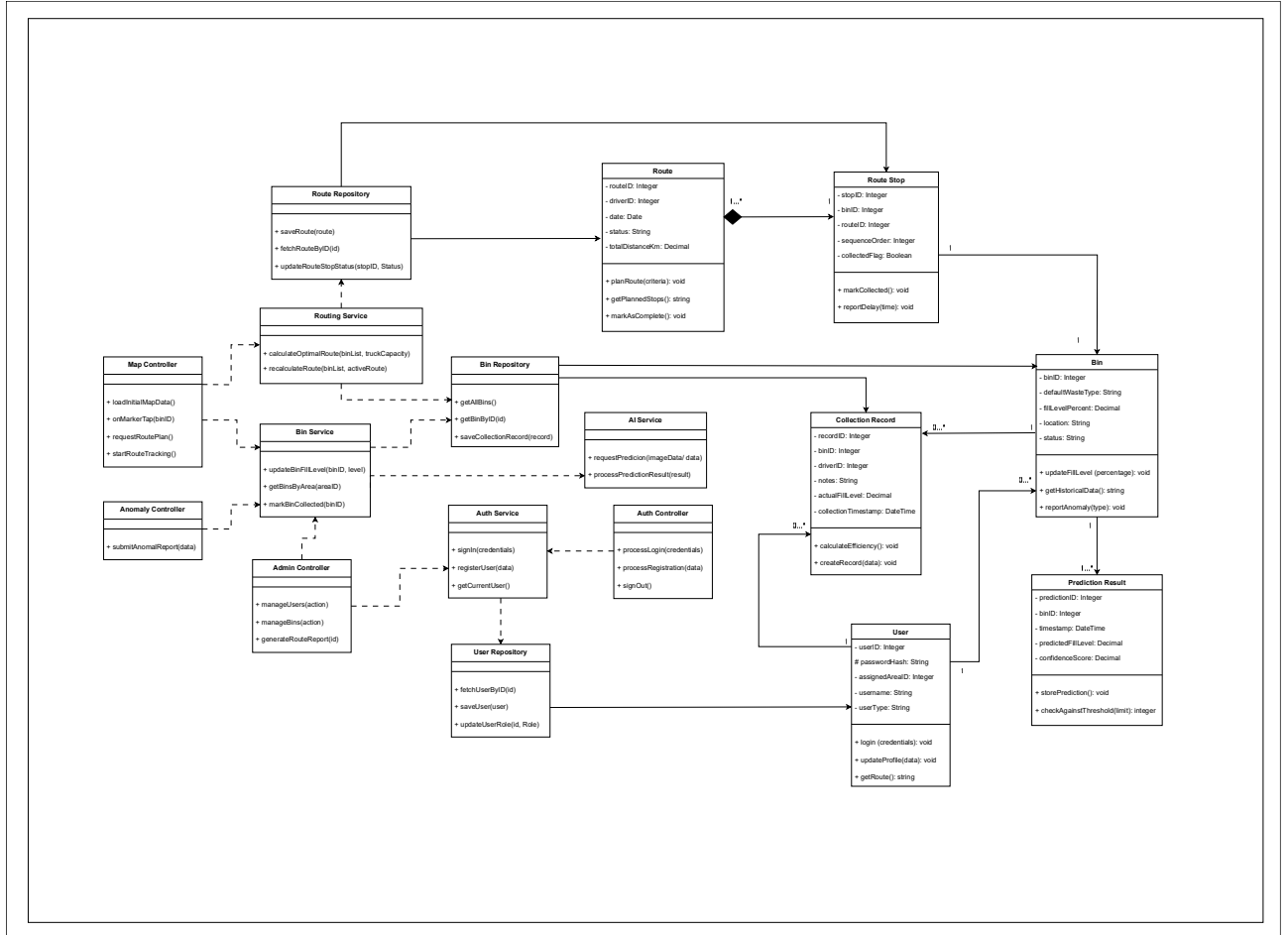


Figure 3.3: Class Diagram of the System

3.4 Interaction Viewpoint

3.4.1 Sequence Diagram for Admin Manage Users

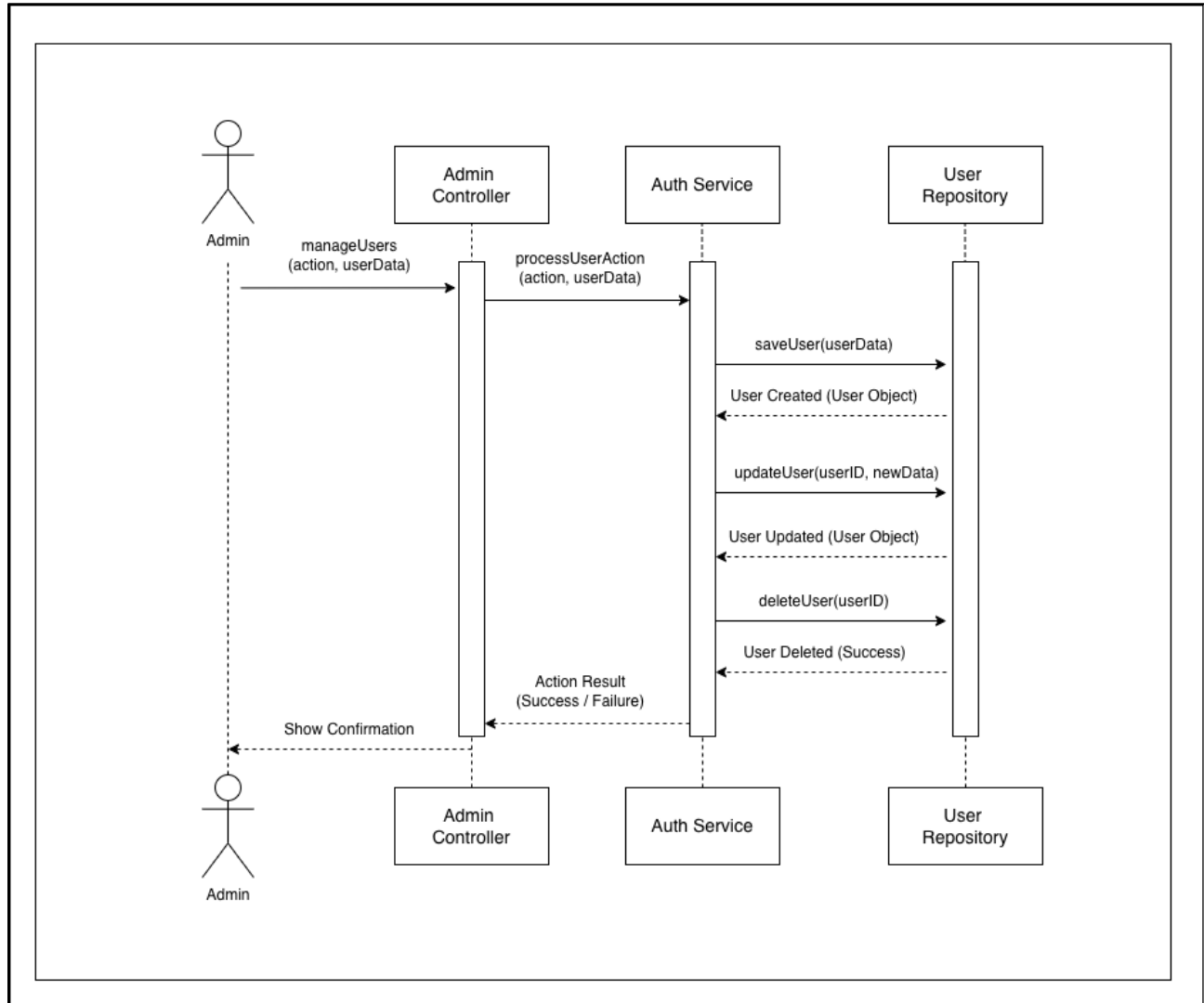


Figure 3.4: Sequence Diagram for Manage Users

3.4.2 Sequence Diagram for Admin Manage Bins

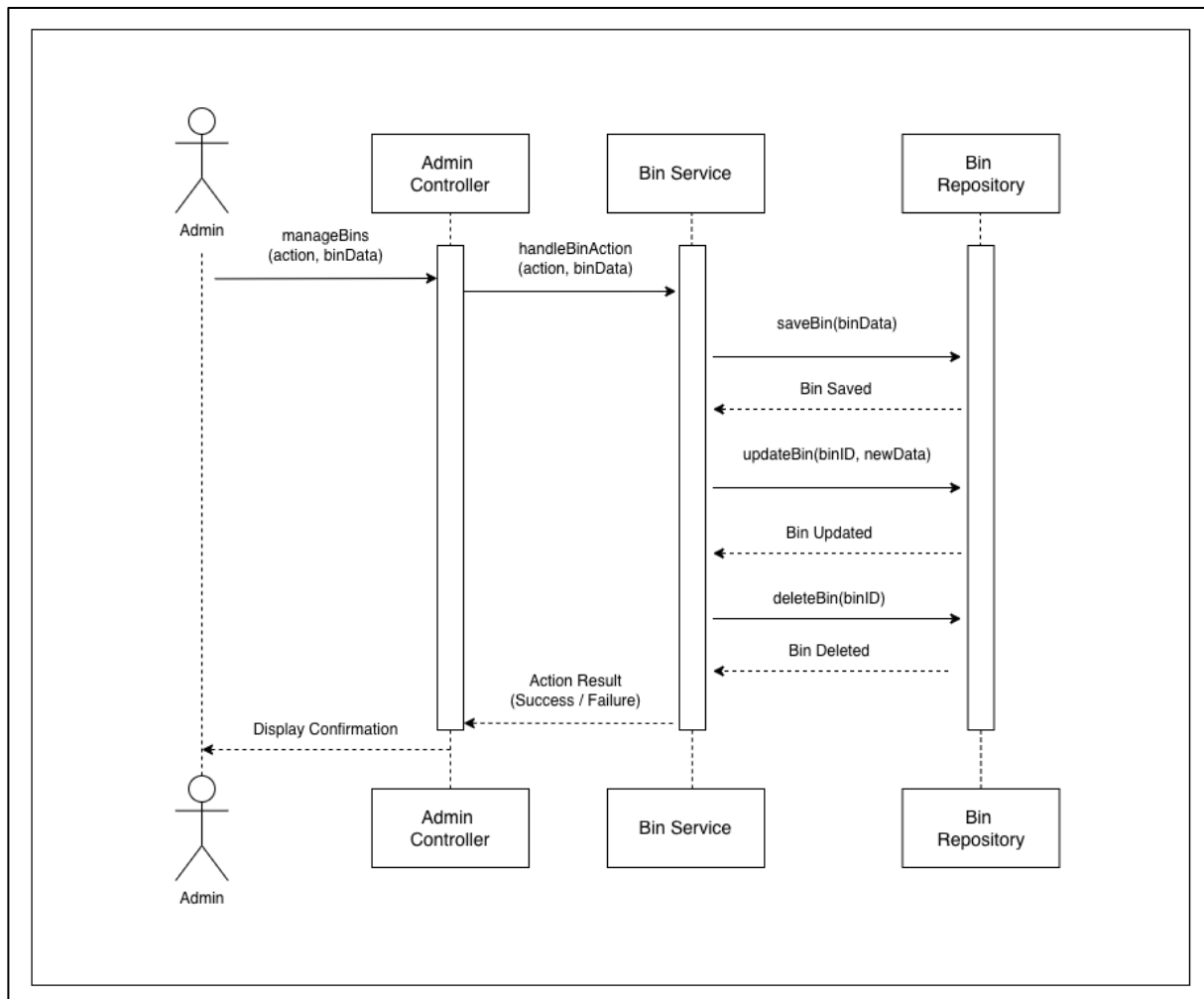


Figure 3.5: Sequence Diagram for Manage Bins

3.4.3 Sequence Diagram for Admin View Dashboard

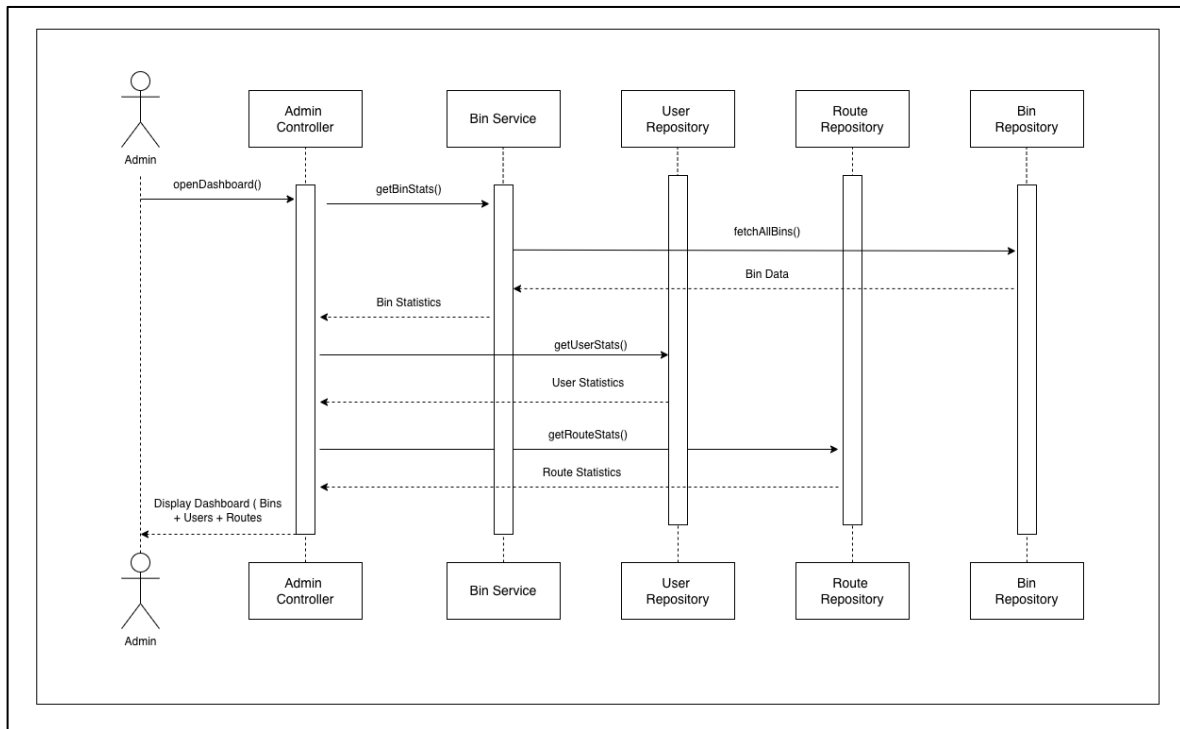


Figure 3.6: Sequence Diagram for View Dashboard

3.4.4 Sequence Diagram for Admin Generate Reports

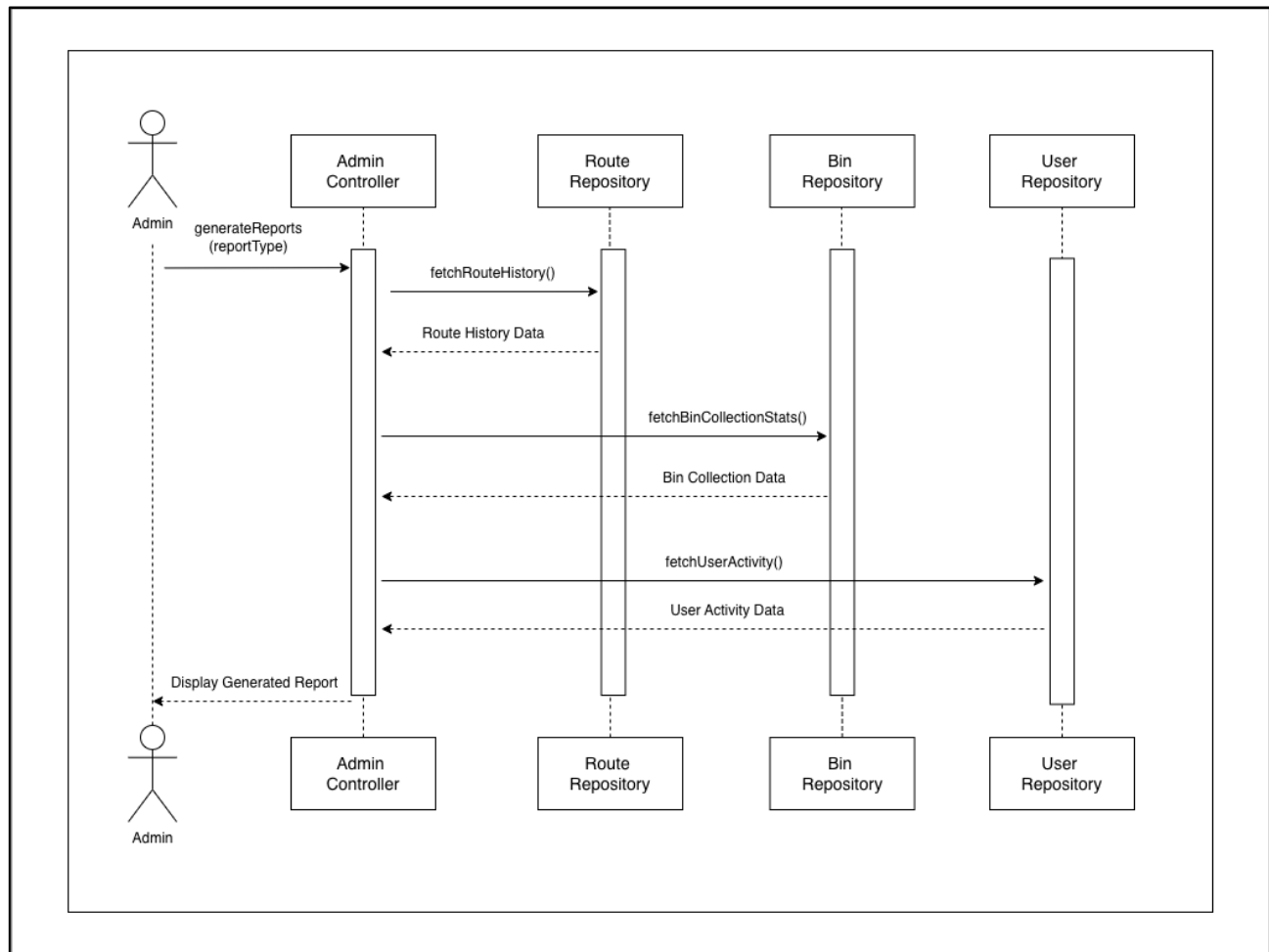


Figure 3.7: Sequence Diagram for Generate Reports

3.4.5 Sequence Diagram for Admin Login

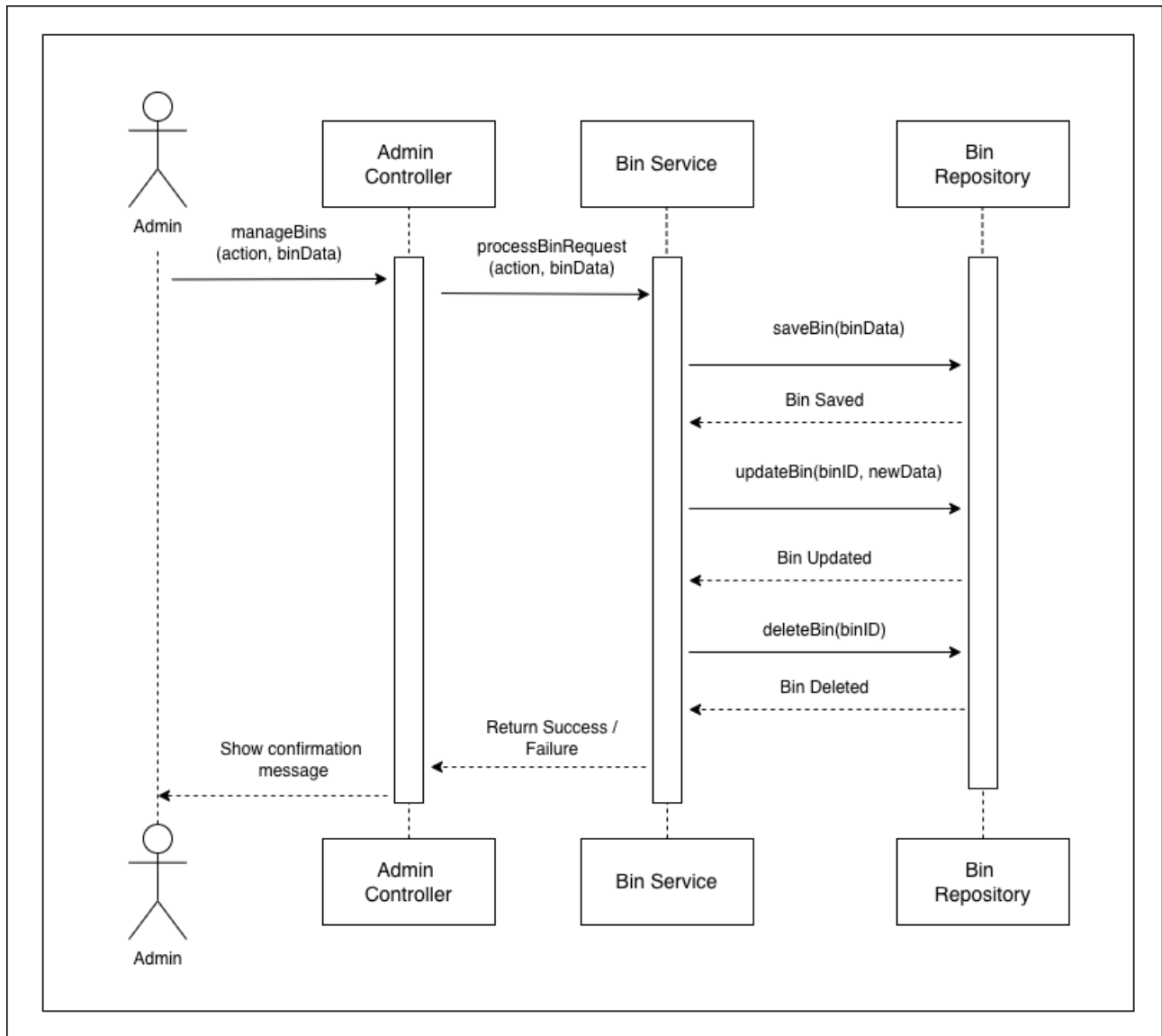


Figure 3.8: Sequence Diagram for Admin Login

3.4.6 Sequence Diagram for Worker Login

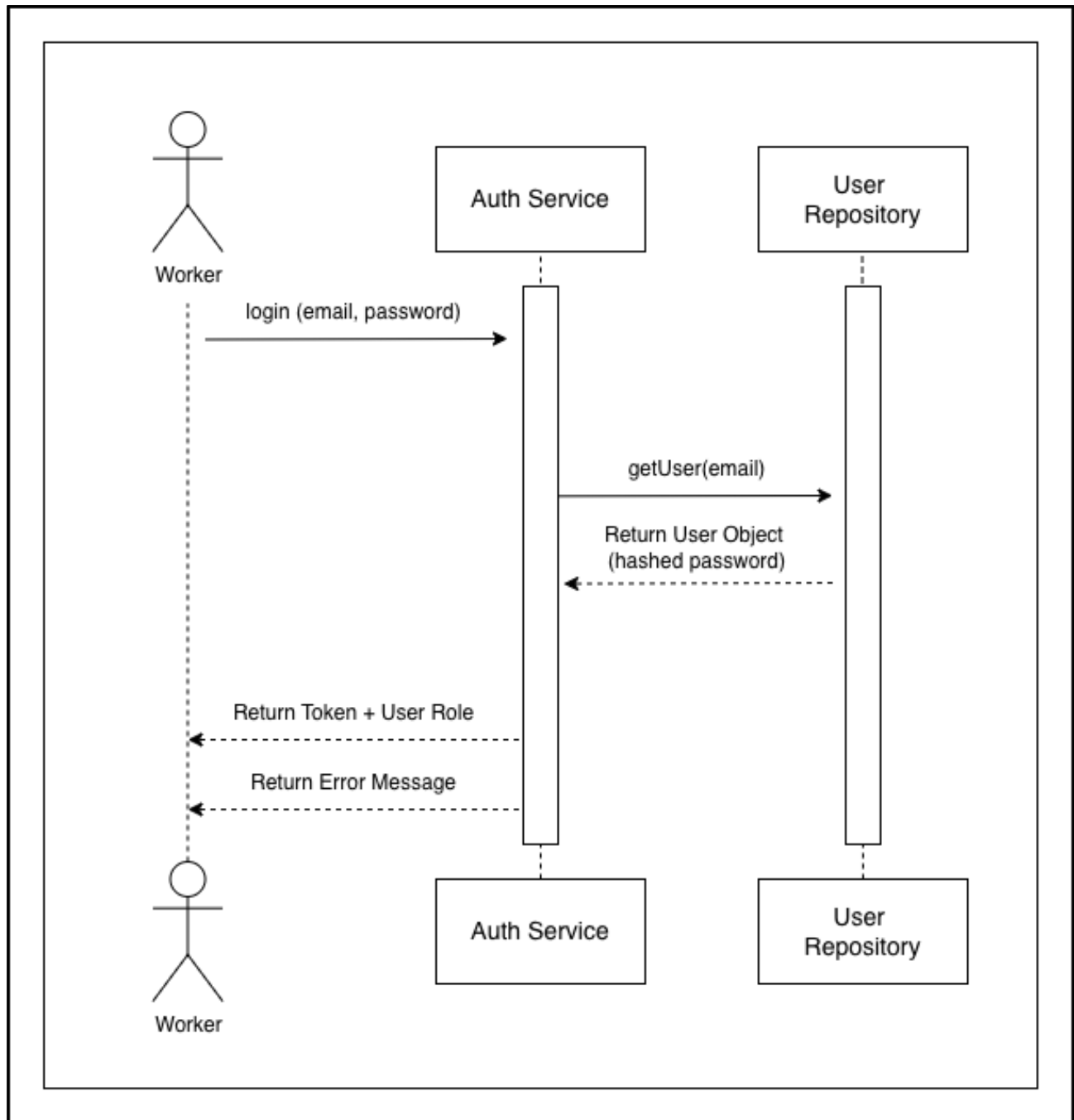


Figure 3.9: Sequence Diagram for Worker Login

3.4.7 Sequence Diagram for Worker Register

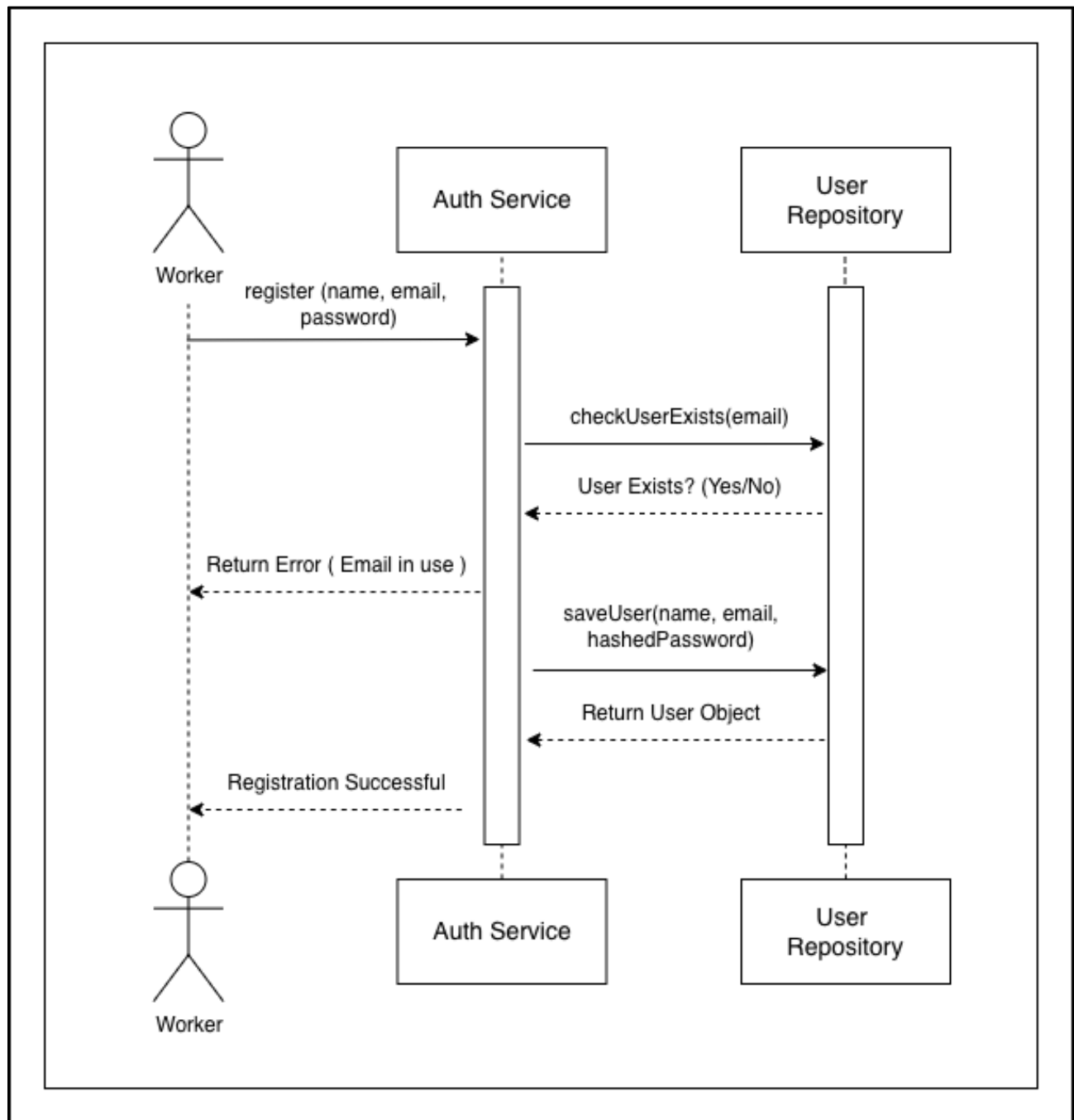


Figure 3.10: Sequence Diagram for Register

3.4.8 Sequence Diagram for Worker View Bin Status

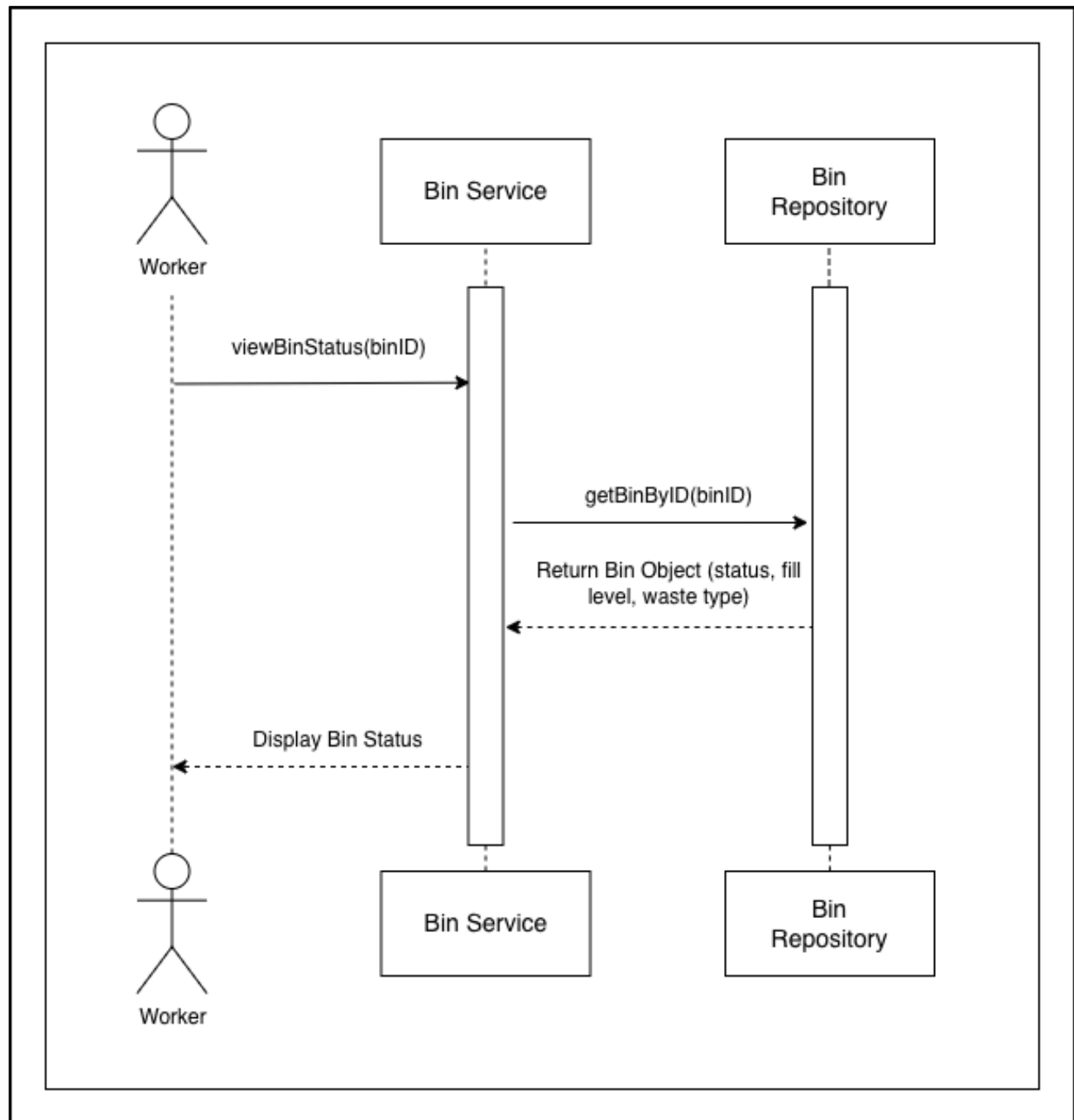


Figure 3.11: Sequence Diagram for View Bin Status

3.4.9 Sequence Diagram for Worker Plan Optimized Route

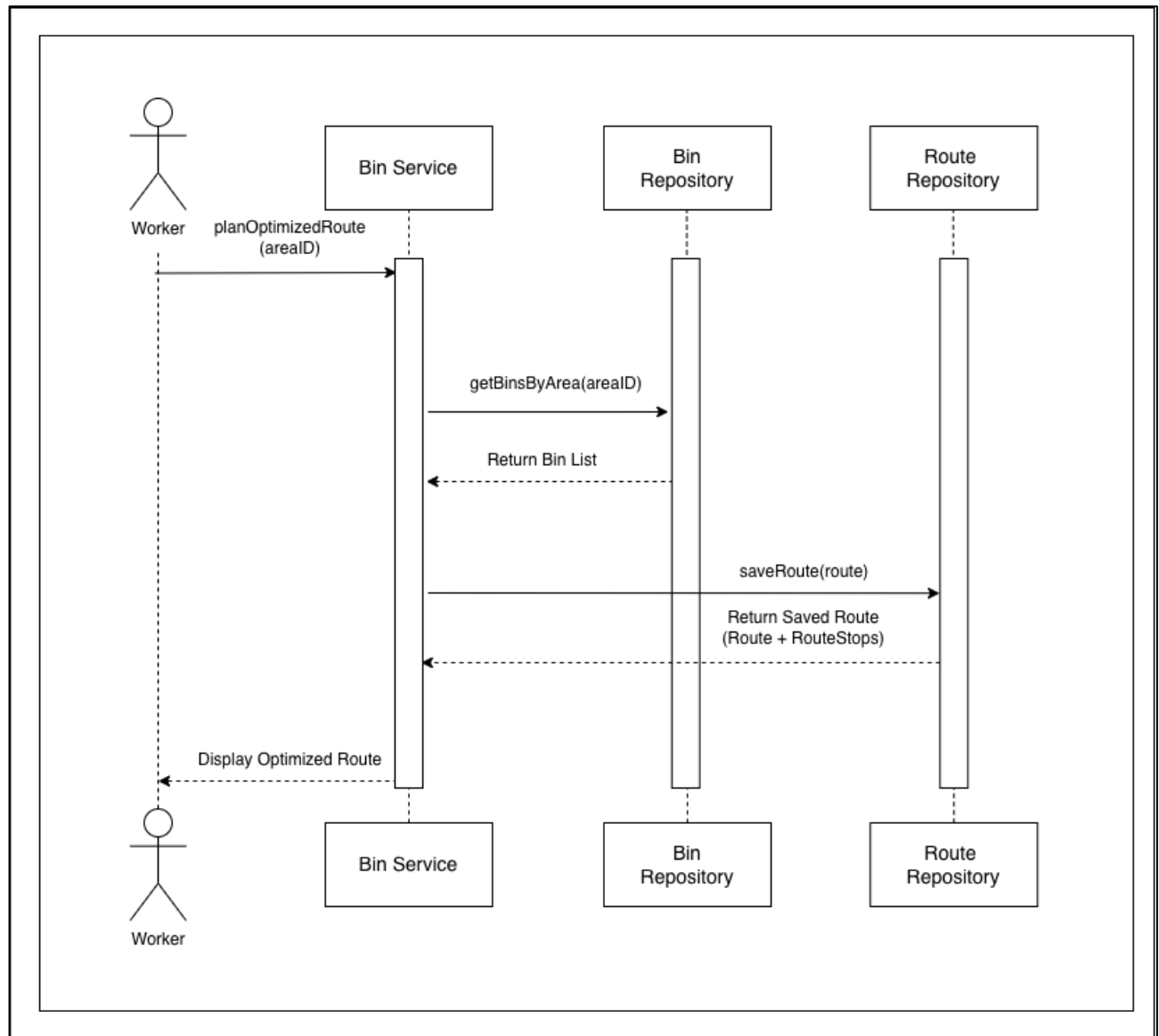


Figure 3.12: Sequence Diagram for Plan Optimized Route

3.4.10 Sequence Diagram for Worker Receive Alerts

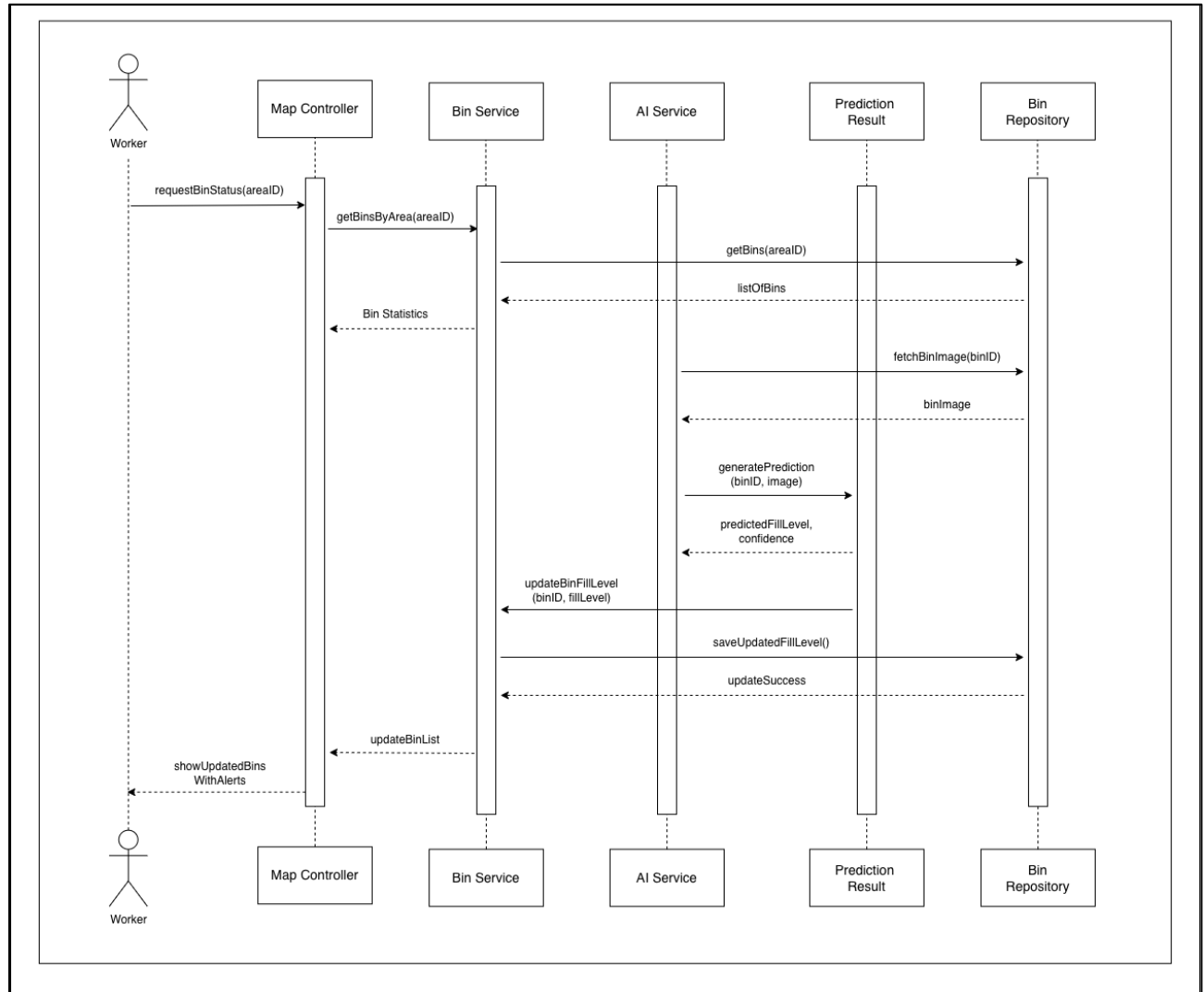


Figure 3.13: Sequence Diagram for Receive Alerts

3.4.11 Sequence Diagram for Worker Mark Bin as Collected

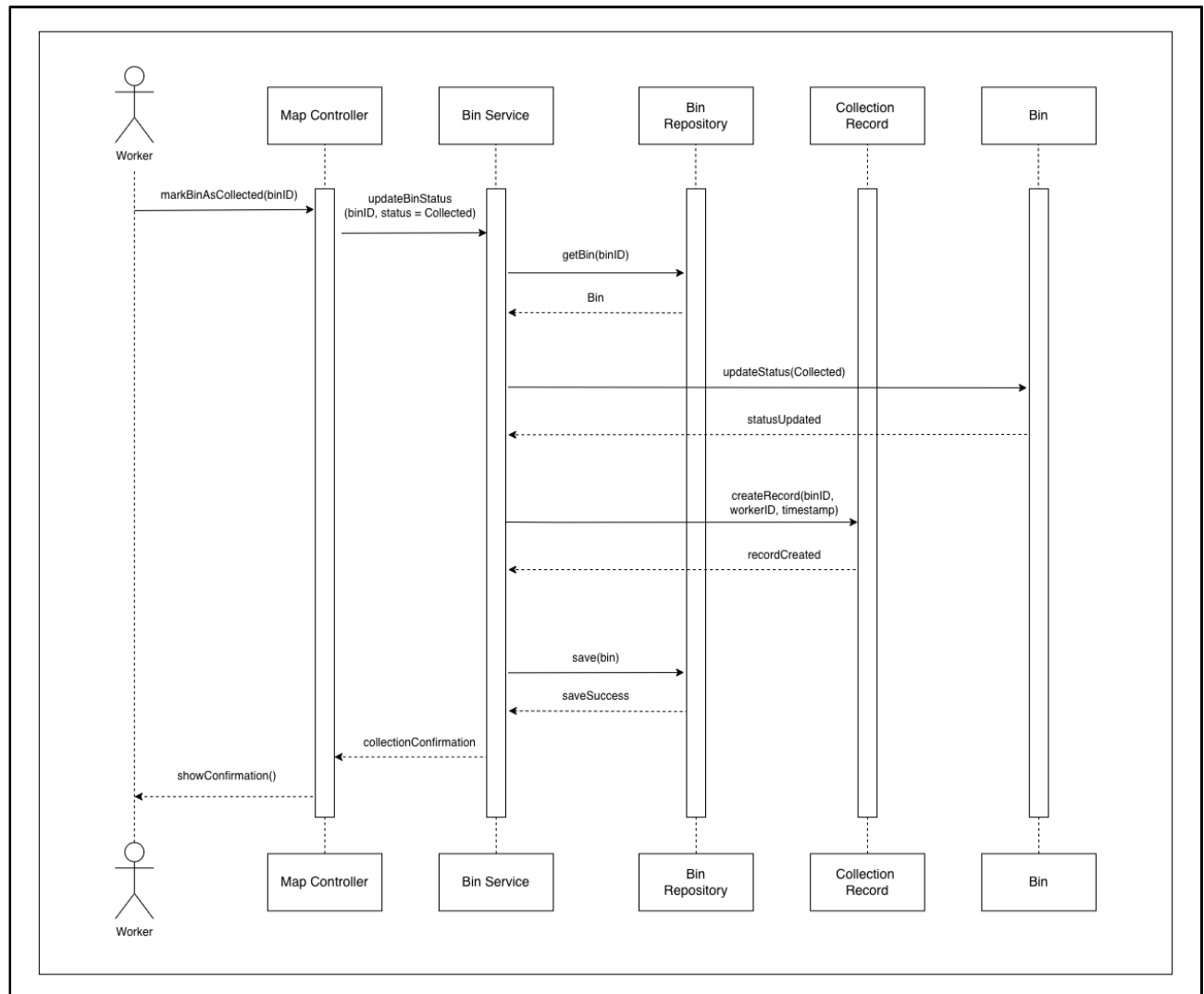


Figure 3.14: Sequence Diagram for Mark Bin as Collected

3.4.12 Sequence Diagram for Worker Report Bin Anomaly

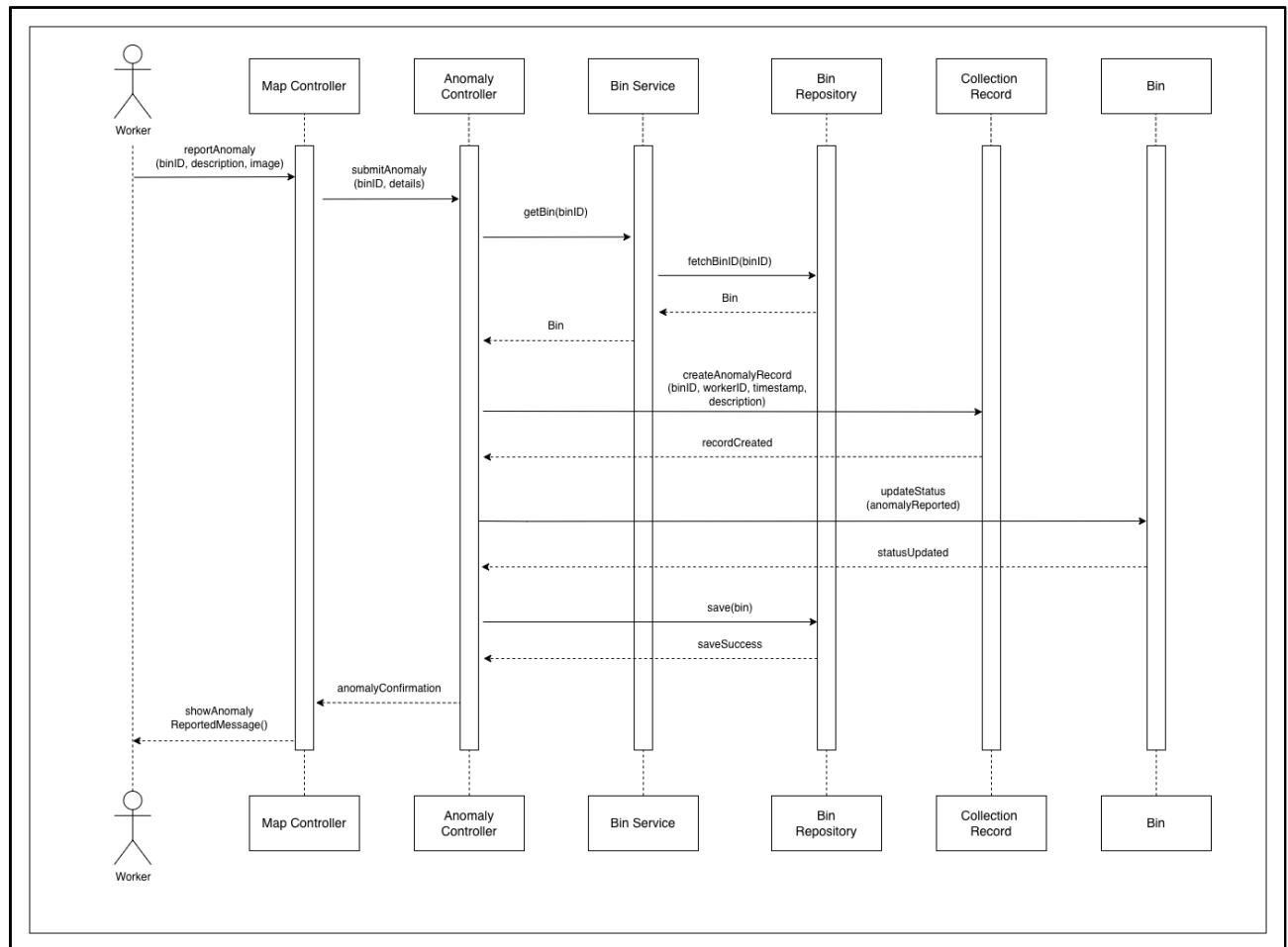


Figure 3.15: Sequence Diagram for Report Bin Anomaly

3.5 State Dynamics Viewpoint

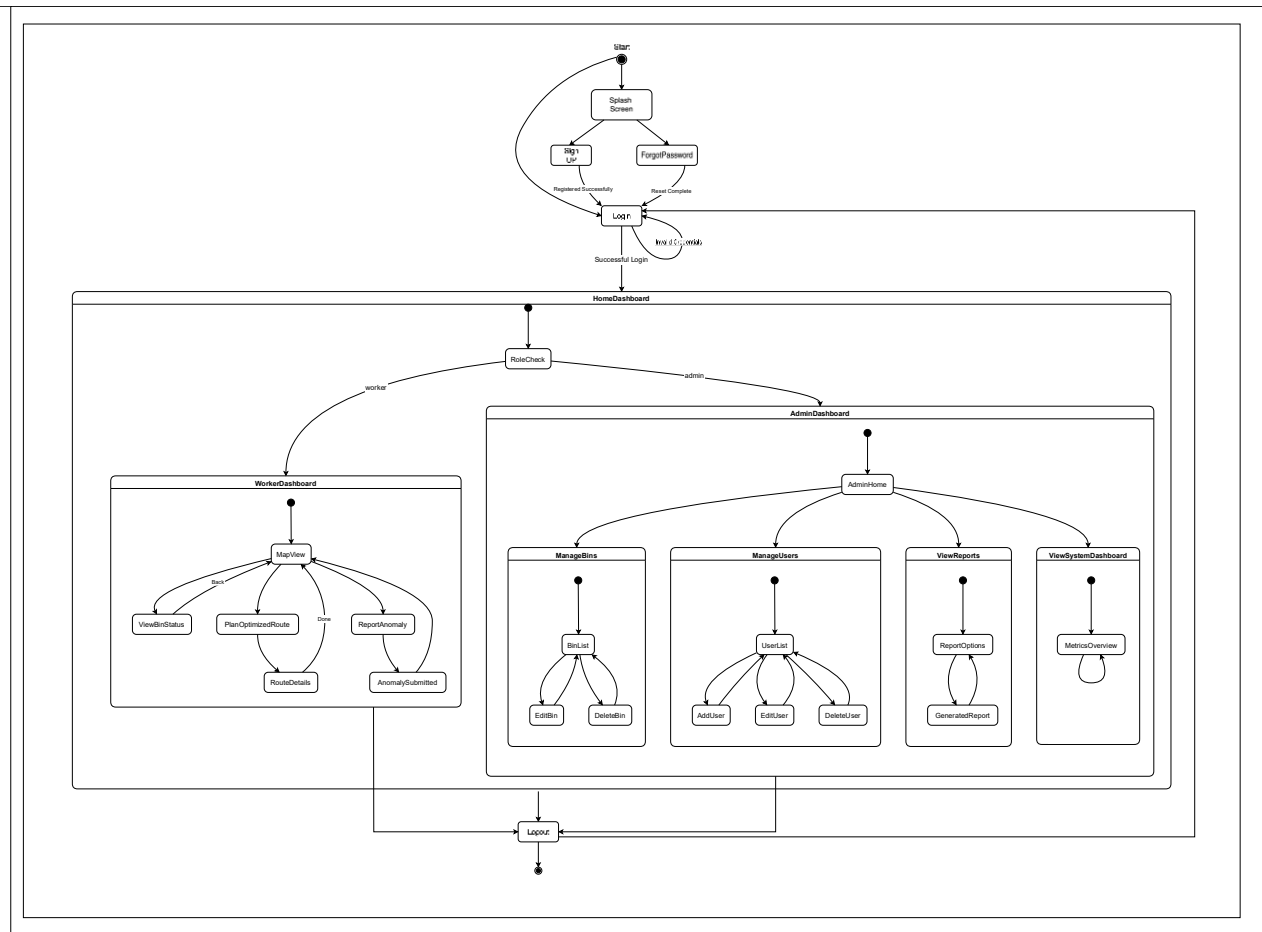


Figure 3.16: State Machine Diagram of The System

3.6 Algorithm Viewpoint

3.6.1 Activity Diagram

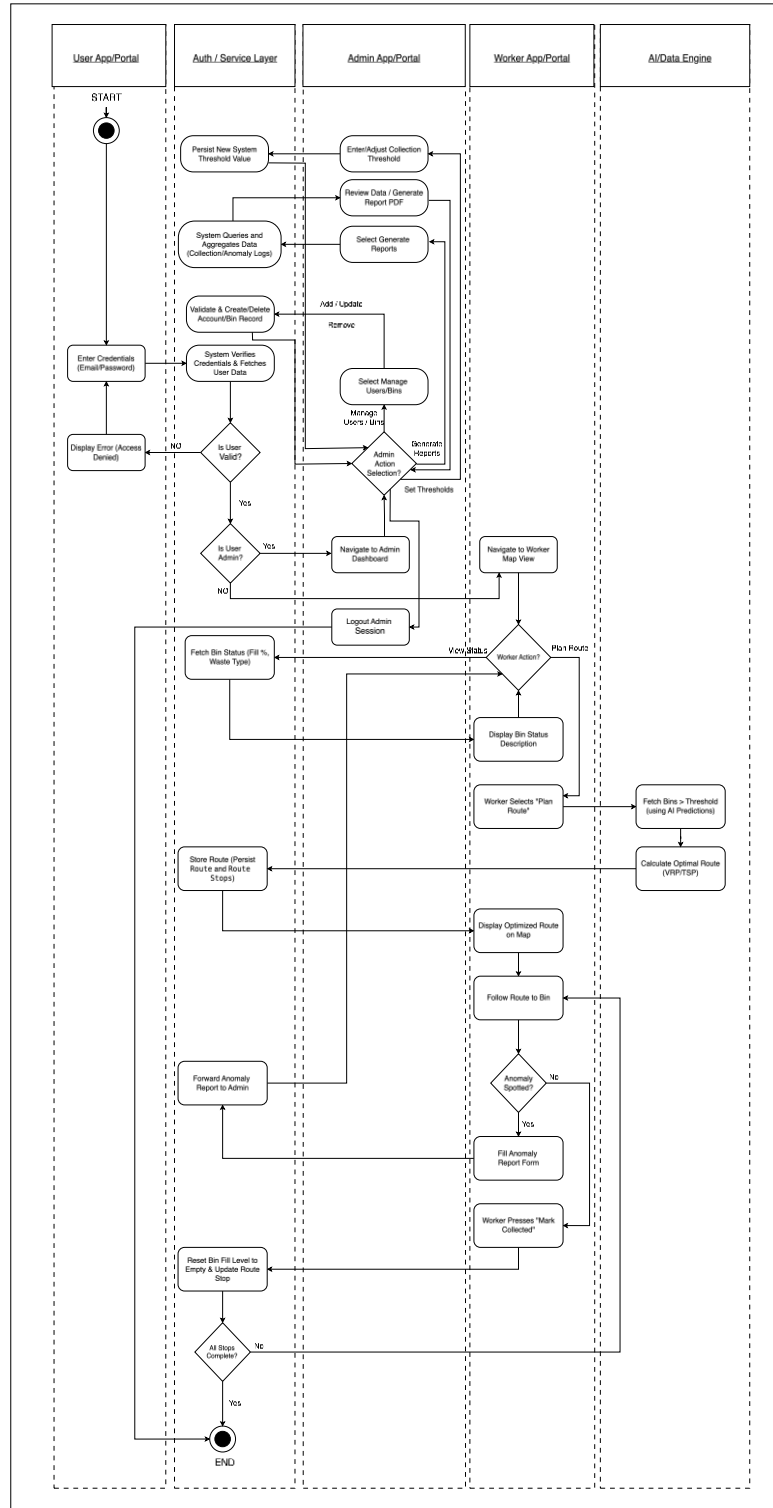


Figure 3.17: Activity Diagram of The System

4 DEVELOPMENT AND TOOLS

This chapter sheds light on the development plan and tools used to develop the Clean Core system. It covers how the workload was divided between team members, gives a walkthrough of all phases of system development, and discusses possible expansion and future updates. Readers will find details about the technologies selected, the rationale behind these choices, and the development methodology followed throughout the project lifecycle.

4.1 Introduction

The Development and Tools chapter covers the technologies, frameworks, and tools used to build Clean Core. This chapter begins with an overview of the development environment, detailing the selected programming languages, frameworks, and databases that form the system's technical foundation. It also outlines the team structure, work distribution, and project timeline that guided the implementation from initial planning through final deployment.

4.2 Development Plan

Clean Core is developed by a team of two members:

- Ayesha Noman (FA22-BCS-009/WAH)
- Ayesha Nadeem (FA22-BCS-020/WAH)

Following is a Gantt Chart displaying the development plan in detail.

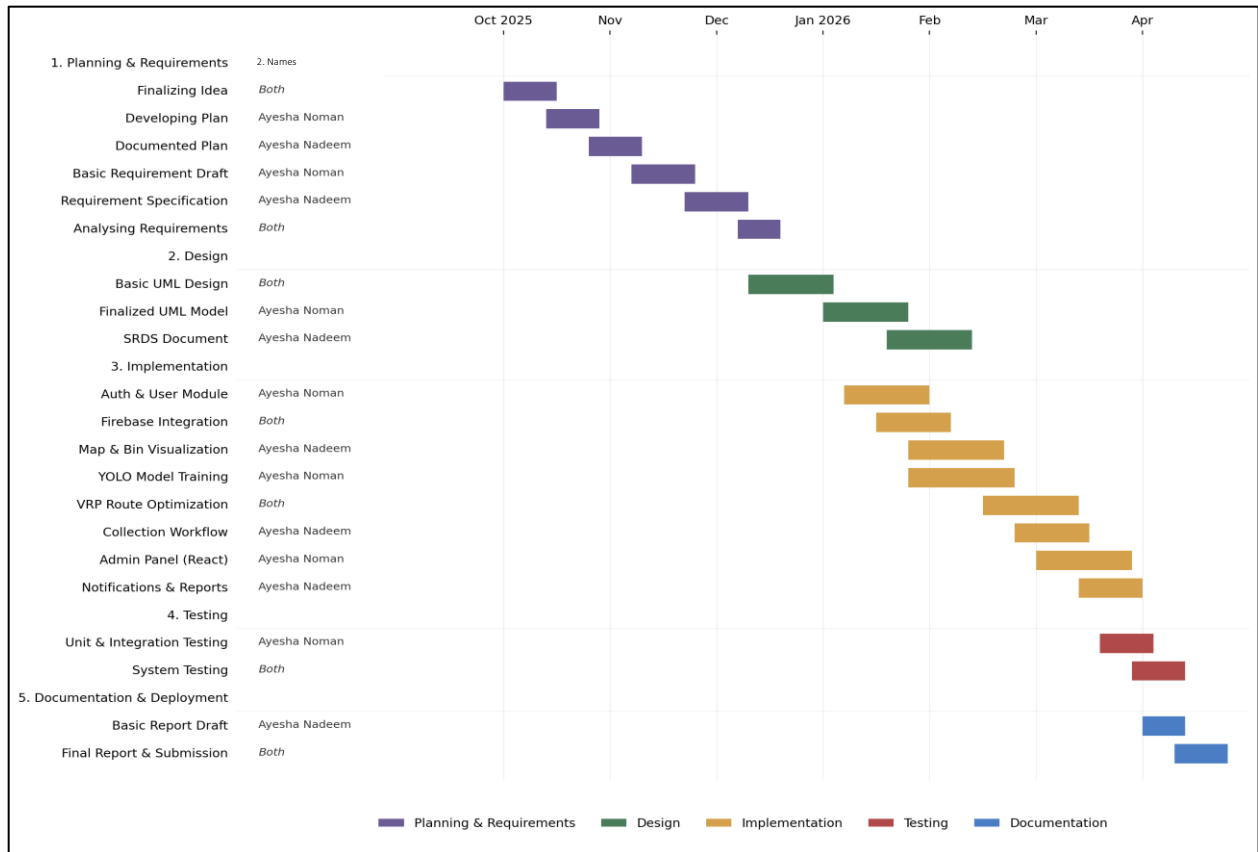


Figure 4.1: Gantt Chart Of Development Plan

4.3 Development Tools

Following development tools are used to develop the system:

- Language: Dart
- Framework: Flutter v3.29.3
- Database: Firebase v13.23.0
- IDE: Visual Studio Code v1.106.2
- Deployment Environment: Android
- Developed on: Laptop (Intel Core i5, 12th Generation).

4.4 Conclusion and Future Work/Extensions

In conclusion, Clean Core provides an intelligent platform for waste collection optimization, offering features that allow drivers to view bin fill levels, receive optimized routes, and mark

collections in real-time. The system's foundational features, including AI-based prediction, VRP route optimization, and real-time tracking, lay the groundwork for smarter urban waste management. The modular architecture ensures maintainability and allows for incremental feature additions.

Looking ahead, we envision Clean Core growing to include more advanced features:

- **Multi-Vehicle Fleet Coordination:** Extend the VRP algorithm to manage multiple trucks simultaneously, distributing workload optimally across the entire fleet.
- **Physical IoT Sensor Integration:** Replace simulated bin data with real IoT sensors (ultrasonic, weight-based) for true real-time fill monitoring.
- **Contamination Detection:** Enhance the YOLO model to detect contamination, hazardous materials, or recyclable content in bins.
- **Predictive Analytics:** Implement machine learning models that forecast fill levels based on historical patterns, weather, and events.
- **Municipal Billing Integration:** Connect with city billing systems for usage-based waste collection fees.
- **Citizen Reporting Portal:** Allow residents to report overflowing bins or request special pickups through a public interface.
- **Carbon Footprint Dashboard:** Track and visualize environmental impact metrics including fuel saved and emissions reduced.

5 QUALITY ASSURANCE

This chapter discusses how the Clean Core application is verified and validated through various testing approaches. First, we provide a Traceability Matrix that maps functional requirements to their corresponding test cases. After that, we describe the detailed test cases used for validation. Through these tests, we confirm that Clean Core meets its intended quality standards, providing a reliable and efficient experience for sanitation workers and administrators.

5.1 Introduction

In the quality assurance phase, which is mainly based on the Test Plan including testing strategies and types of testing applied to ensure the reliability and accuracy of the application, we give the user a great experience. The Quality Assurance chapter highlights the strategies and processes used to ensure the reliability, performance, and correctness of the Clean Core platform. Through these tests, we confirm that Clean Core meets its intended quality standards.

5.2 Traceability Matrix

The Traceability Matrix maps the functional requirements of Clean Core against the designed test cases. All implemented requirements have been tested and verified. The matrix ensures complete coverage of all functional requirements.

Table 5.1: Traceability Matrix (TC-001 - TC-013)

SFR ID	TC-001	TC-002	TC-003	TC-004	TC-005	TC-006	TC-007	TC-008	TC-009	TC-010	TC-011	TC-012	TC-013	# TC
SFR-001	X													1
SFR-002		X												1
SFR-003			X											1
SFR-004				X										1
SFR-005					X									1
SFR-006						X								1
SFR-007							X							1
SFR-008								X						1
SFR-009									X					1
SFR-010										X				1

SFR ID	TC-001	TC-002	TC-003	TC-004	TC-005	TC-006	TC-007	TC-008	TC-009	TC-010	TC-011	TC-012	TC-013	# TC
SFR-011											X			1
SFR-012												X		1
SFR-013													X	1
SFR-014														1
SFR-015														1
SFR-016														1
SFR-017														1
SFR-018														1
SFR-019														1
SFR-020														1
SFR-021														1
SFR-022														1
SFR-023														1
SFR-024														1
SFR-025														1

Table 5.2: Traceability Matrix (TC-014 - TC-025)

SFR ID	TC-014	TC-015	TC-016	TC-017	TC-018	TC-019	TC-020	TC-021	TC-022	TC-023	TC-024	TC-025	# TC
SFR-001													1
SFR-002													1
SFR-003													1
SFR-004													1
SFR-005													1
SFR-006													1
SFR-007													1
SFR-008													1
SFR-009													1
SFR-010													1
SFR-011													1
SFR-012													1
SFR-013													1
SFR-014	X												1
SFR-015		X											1
SFR-016			X										1
SFR-017				X									1
SFR-018					X								1
SFR-019						X							1
SFR-020							X						1
SFR-021								X					1
SFR-022									X				1
SFR-023										X			1
SFR-024											X		1
SFR-025												X	1

5.3 Test Plan

Clean Core was tested throughout all phases of development. During implementation, core modules such as authentication, map visualization, route optimization, and bin management were tested individually through Unit Testing. Integration Testing ensured smooth interaction between modules.

Table 5.3: Test Case TC-001 - User Registration

Test Case ID	TC-001
Functional Req. ID	SFR-001
Test Case Name	User Registration
Description	Register a new worker with valid details
Input	Name: Test Worker, Email: worker@test.com, Password: Test@123
Expected Output	Account created successfully; verification email sent; redirect to login page
Actual Output	Account created successfully; verification email sent; redirect to login page
Status	PASSED

Table 5.4: Test Case TC-002 - User Login

Test Case ID	TC-002
Functional Req. ID	SFR-002
Test Case Name	User Login
Description	Login with valid credentials
Input	Email: worker@test.com, Password: Test@123
Expected Output	User authenticated; session created; redirect to home page
Actual Output	User authenticated; session created; redirect to home page
Status	PASSED

Table 5.5: Test Case TC-003 - Profile Management

Test Case ID	TC-003
Functional Req. ID	SFR-003
Test Case Name	Profile Management
Description	Update user profile information
Input	New Name: Updated Name, New Phone: 03001234567
Expected Output	Profile updated successfully; changes reflected in database
Actual Output	Profile updated successfully; changes reflected in database
Status	PASSED

Table 5.6: Test Case TC-004 - Password Reset

Test Case ID	TC-004
Functional Req. ID	SFR-004
Test Case Name	Password Reset
Description	Reset password via email verification
Input	Email: worker@test.com
Expected Output	Password reset email sent; link opens reset page
Actual Output	Password reset email sent; link opens reset page
Status	PASSED

Table 5.7: Test Case TC-005 - Session Management

Test Case ID	TC-005
Functional Req. ID	SFR-005
Test Case Name	Session Management
Description	Maintain user session until logout or timeout
Input	User logs in and remains inactive for 30 minutes
Expected Output	Session maintained; auto logout after timeout period
Actual Output	Session maintained; auto logout after timeout period
Status	PASSED

Table 5.8: Test Case TC-006 - Bin Simulation

Test Case ID	TC-006
Functional Req. ID	SFR-006
Test Case Name	Bin Simulation
Description	Generate and update bin fill levels at regular intervals
Input	System runs for 15 minutes
Expected Output	Fill levels updated for all bins in database
Actual Output	Fill levels updated for all bins in database
Status	PASSED

Table 5.9: Test Case TC-007 - AI Fill Prediction

Test Case ID	TC-007
Functional Req. ID	SFR-007
Test Case Name	AI Fill Prediction
Description	YOLO model predicts bin fill level from image
Input	Upload bin image to prediction API
Expected Output	Returns class (Empty/Low/Medium/Full) with confidence score within 2 seconds
Actual Output	Returns class with confidence score in 1.8 seconds
Status	PASSED

Table 5.10: Test Case TC-008 - Bin Data Storage

Test Case ID	TC-008
Functional Req. ID	SFR-008
Test Case Name	Bin Data Storage
Description	Store bin data in Firestore
Input	Add new bin with location and fill level
Expected Output	Data stored successfully; retrievable via query
Actual Output	Data stored successfully; retrievable via query
Status	PASSED

Table 5.11: Test Case TC-009 - Real-time Bin Status

Test Case ID	TC-009
Functional Req. ID	SFR-009
Test Case Name	Real-time Bin Status
Description	Bin status freshness check
Input	Request bin data from mobile app
Expected Output	Data retrieved with timestamp less than 15 minutes old
Actual Output	Data retrieved with timestamp less than 15 minutes old
Status	PASSED

Table 5.12: Test Case TC-010 - Interactive Map Display

Test Case ID	TC-010
Functional Req. ID	SFR-010
Test Case Name	Interactive Map Display
Description	Display map with bin markers and driver location
Input	Navigate to Map tab; GPS enabled
Expected Output	Map loads within 3 seconds; color-coded markers displayed
Actual Output	Map loads in 2.5 seconds; markers displayed correctly
Status	PASSED

Table 5.13: Test Case TC-011 - Route Planning

Test Case ID	TC-011
Functional Req. ID	SFR-011
Test Case Name	Route Planning
Description	Initiate route planning using high-priority bins
Input	Tap 'Plan Route' with 10 bins above threshold
Expected Output	High-priority bins selected; planning UI displayed
Actual Output	High-priority bins selected; planning UI displayed
Status	PASSED

Table 5.14: Test Case TC-012 - Route Optimization

Test Case ID	TC-012
Functional Req. ID	SFR-012
Test Case Name	Route Optimization
Description	Calculate optimal route using VRP algorithm
Input	10 bins with varying fill levels; driver at starting location
Expected Output	Optimized route calculated within 5 seconds
Actual Output	Route calculated in 2.3 seconds; Full bins prioritized
Status	PASSED

Table 5.15: Test Case TC-013 - Route Visualization

Test Case ID	TC-013
Functional Req. ID	SFR-013
Test Case Name	Route Visualization
Description	Display optimized route on map
Input	Route generated from previous test
Expected Output	Route displayed as path with numbered waypoints
Actual Output	Route displayed as path with numbered waypoints
Status	PASSED

Table 5.16: Test Case TC-014 - Mark as Collected

Test Case ID	TC-014
Functional Req. ID	SFR-014
Test Case Name	Mark as Collected
Description	Mark a bin as collected after emptying
Input	Tap bin marker; tap 'Mark as Collected'
Expected Output	Bin status updated to Empty; timestamp recorded; removed from route
Actual Output	Bin status updated; timestamp recorded; removed from route
Status	PASSED

Table 5.17: Test Case TC-015 - Report Anomaly

Test Case ID	TC-015
Functional Req. ID	SFR-015
Test Case Name	Report Anomaly
Description	Report bin anomaly with predefined reason
Input	Tap bin; select 'Blocked'; add note; submit
Expected Output	Anomaly report created; admin notified
Actual Output	Anomaly report created; admin notified
Status	PASSED

Table 5.18: Test Case TC-016 - Real-time Route Updates

Test Case ID	TC-016
Functional Req. ID	SFR-016
Test Case Name	Real-time Route Updates
Description	Update route display as bins are collected
Input	Mark 3 out of 5 bins as collected
Expected Output	Route updates in real-time; remaining stops shown
Actual Output	Route updates in real-time; remaining stops shown
Status	PASSED

Table 5.19: Test Case TC-017 - Push Notifications

Test Case ID	TC-017
Functional Req. ID	SFR-017
Test Case Name	Push Notifications
Description	Receive push notification for route assignment
Input	Admin assigns new route to worker
Expected Output	Worker receives notification on mobile device
Actual Output	Worker receives notification within 1 second
Status	PASSED

Table 5.20: Test Case TC-018 - Admin Dashboard

Test Case ID	TC-018
Functional Req. ID	SFR-018
Test Case Name	Admin Dashboard
Description	View real-time operational status
Input	Admin logs in and navigates to dashboard
Expected Output	Dashboard displays active drivers and collection progress
Actual Output	Dashboard displays active drivers and collection progress
Status	PASSED

Table 5.21: Test Case TC-019 - Anomaly Management

Test Case ID	TC-019
Functional Req. ID	SFR-019
Test Case Name	Anomaly Management
Description	View and filter unresolved anomalies
Input	Admin opens anomalies page; applies filter
Expected Output	Filtered anomaly list displayed with location and timestamp
Actual Output	Filtered anomaly list displayed correctly
Status	PASSED

Table 5.22: Test Case TC-020 - Performance Reports

Test Case ID	TC-020
Functional Req. ID	SFR-020
Test Case Name	Performance Reports
Description	Generate performance report for date range
Input	Select date range; click 'Generate Report'
Expected Output	Report generated showing mileage, time, and bins collected
Actual Output	Report generated showing mileage, time, and bins collected
Status	PASSED

Table 5.23: Test Case TC-021 - User Management

Test Case ID	TC-021
Functional Req. ID	SFR-021
Test Case Name	User Management
Description	Admin creates new worker account
Input	Enter worker details; assign role; click 'Create'
Expected Output	User account created; credentials sent via email
Actual Output	User account created; credentials sent via email
Status	PASSED

Table 5.24: Test Case TC-022 - Bin Management

Test Case ID	TC-022
Functional Req. ID	SFR-022
Test Case Name	Bin Management
Description	Admin adds a new bin to the system
Input	Enter bin location and zone; click 'Add Bin'
Expected Output	Bin added to database; visible on map
Actual Output	Bin added to database; visible on map
Status	PASSED

Table 5.25: Test Case TC-023 - Zone Assignment

Test Case ID	TC-023
Functional Req. ID	SFR-023
Test Case Name	Zone Assignment
Description	Assign collection zone to a driver
Input	Select driver; select zone; click 'Assign'
Expected Output	Zone assigned; driver sees bins in assigned zone
Actual Output	Zone assigned; driver sees bins in assigned zone
Status	PASSED

Table 5.26: Test Case TC-024 - Time Slot Scheduling

Test Case ID	TC-024
Functional Req. ID	SFR-024
Test Case Name	Time Slot Scheduling
Description	Admin sets collection time slot
Input	Create new time slot: Morning 8AM-12PM
Expected Output	Time slot saved and applied to workers
Actual Output	Time slot saved and applied to workers
Status	PASSED

Table 5.27: Test Case TC-025 - Historical Data Access

Test Case ID	TC-025
Functional Req. ID	SFR-025
Test Case Name	Historical Data Access
Description	View and download historical collection data
Input	Select date range; click 'Export'
Expected Output	Historical data exported as CSV file
Actual Output	Historical data exported as CSV file
Status	PASSED

6 USER MANUAL

The User Manual chapter serves as a comprehensive guide for understanding and using the Clean Core application. It includes hardware and software requirements, installation instructions, and detailed operating guidelines for all functionalities. This manual is intended for sanitation workers (drivers) who will use the mobile application and administrators who will manage the system through the web panel.

6.1 Introduction

This chapter provides a comprehensive overview of Clean Core, starting with its key features and how they support efficient waste collection. It outlines the hardware and software requirements for optimal performance, followed by a step-by-step installation guide. The operating manual offers detailed instructions on user account management, map navigation, route planning, bin collection, and anomaly reporting for workers, as well as dashboard operations for administrators.

6.2 Hardware/Software Requirements for the System

Hardware Requirements (Mobile App):

- Processor: Quad-core 1.5 GHz or better
- RAM: Minimum 4 GB
- Storage: Minimum 200 MB free space
- GPS: Functional GPS hardware for location tracking
- Network: Stable internet connection (4G/LTE/Wi-Fi)

Software Requirements (Mobile App):

- Android: Version 10.0 or higher
- iOS: Version 14.0 or higher

Requirements (Admin Panel - Web):

- Browser: Chrome, Firefox, Edge, or Safari (latest versions)
- Display: Minimum resolution 1280 x 1024 pixels
- Internet: Stable broadband connection

6.3 Installation guide for Application

Step 1: Clone the Repository

- `git clone https://github.com/ayeshanoman/clean-core.git`
- `cd clean-core`

Step 2: Flutter Mobile App Setup

2.1 Install Flutter SDK:

- Download Flutter SDK from <https://flutter.dev/docs/get-started/install>
- Extract and add flutter/bin to system PATH
- `flutter --version`

2.2 Verify Flutter installation:

- `flutter doctor`
- `flutter doctor --android-licenses`

2.3 Install Flutter dependencies:

- `cd mobile_app`
- `flutter pub get`

2.4 Configure Firebase for the app:

- `dart pub global activate flutterfire_cli`
- `flutterfire configure`
- Place google-services.json in android/app/ directory

2.5 Configure Google Maps API key:

- Get API key from <https://console.cloud.google.com>
- Enable Maps SDK for Android and Directions API
- Add key to android/app/src/main/AndroidManifest.xml under meta-data tag

2.6 Run the Flutter app:

- flutter run
- # Build release APK
- flutter build apk --release

Step 3: YOLO AI Model Backend Setup (Python)

3.1 Create a Python virtual environment:

- cd ai_backend
- python -m venv venv
- source venv/bin/activate # Linux/macOS
- # venv\Scripts\activate # Windows

3.2 Install Python dependencies:

- pip install --upgrade pip
- pip install -r requirements.txt

3.3 Install YOLOv8 and core libraries:

- pip install ultralytics
- pip install torch torchvision
- pip install fastapi uvicorn pillow opencv-python numpy

3.4 Verify PyTorch installation:

- python -c "import torch; print(torch.__version__)"
- python -c "from ultralytics import YOLO; print('YOLO ready')"

3.5 Place the trained YOLO model:

- Copy best.pt file into ai_backend/models/ directory
- # The best.pt file contains trained weights for bin fill level classification

3.6 Configure environment variables:

- cp .env.example .env
- Edit .env and fill in Firebase credentials, model path, and port configuration

3.7 Start the FastAPI server:

- uvicorn main:app --host 0.0.0.0 --port 8000 --reload
- # API accessible at http://localhost:8000
- # Swagger docs at http://localhost:8000/docs

Step 4: Firebase Configuration

4.1 Create Firebase project:

- Go to <https://console.firebase.google.com>
- Click 'Add Project' and name it 'Clean Core'
- Register Android app with package name com.example.cleancore

4.2 Enable Firebase services:

- Enable Authentication (Email/Password method)
- Create Firestore Database in production mode
- Enable Cloud Storage for profile pictures
- Enable Cloud Messaging for push notifications

4.3 Configure Firestore security rules:

- Navigate to Firestore → Rules tab
- Paste role-based security rules from firestore.rules file
- Click 'Publish' to apply rules

Step 5: Admin Panel Setup (React)

5.1 Install Node.js:

- Download Node.js v18 or higher from <https://nodejs.org>
- `node --version`
- `npm --version`

5.2 Install admin panel dependencies:

- `cd admin_panel`
- `npm install`

5.3 Configure environment variables:

- `cp .env.example .env`
- Edit `.env` and set Firebase configuration and API URL
- `REACT_APP_API_URL=http://localhost:8000`

5.4 Run the admin panel:

- `npm start`
- # Admin panel accessible at `http://localhost:3000`

5.5 Build for production:

- `npm run build`

Step 6: Launch the Application

- Open three separate terminal windows:
 - **Terminal 1 — YOLO AI Backend:**

```
cd ai_backend
source venv/bin/activate
uvicorn main:app --host 0.0.0.0 --port 8000
```
 - **Terminal 2 — React Admin Panel:**

```
cd admin_panel  
npm start
```

• **Terminal 3 — Flutter Mobile App:**

```
cd mobile_app  
flutter run  
# OR build APK for installation  
flutter build apk --release
```

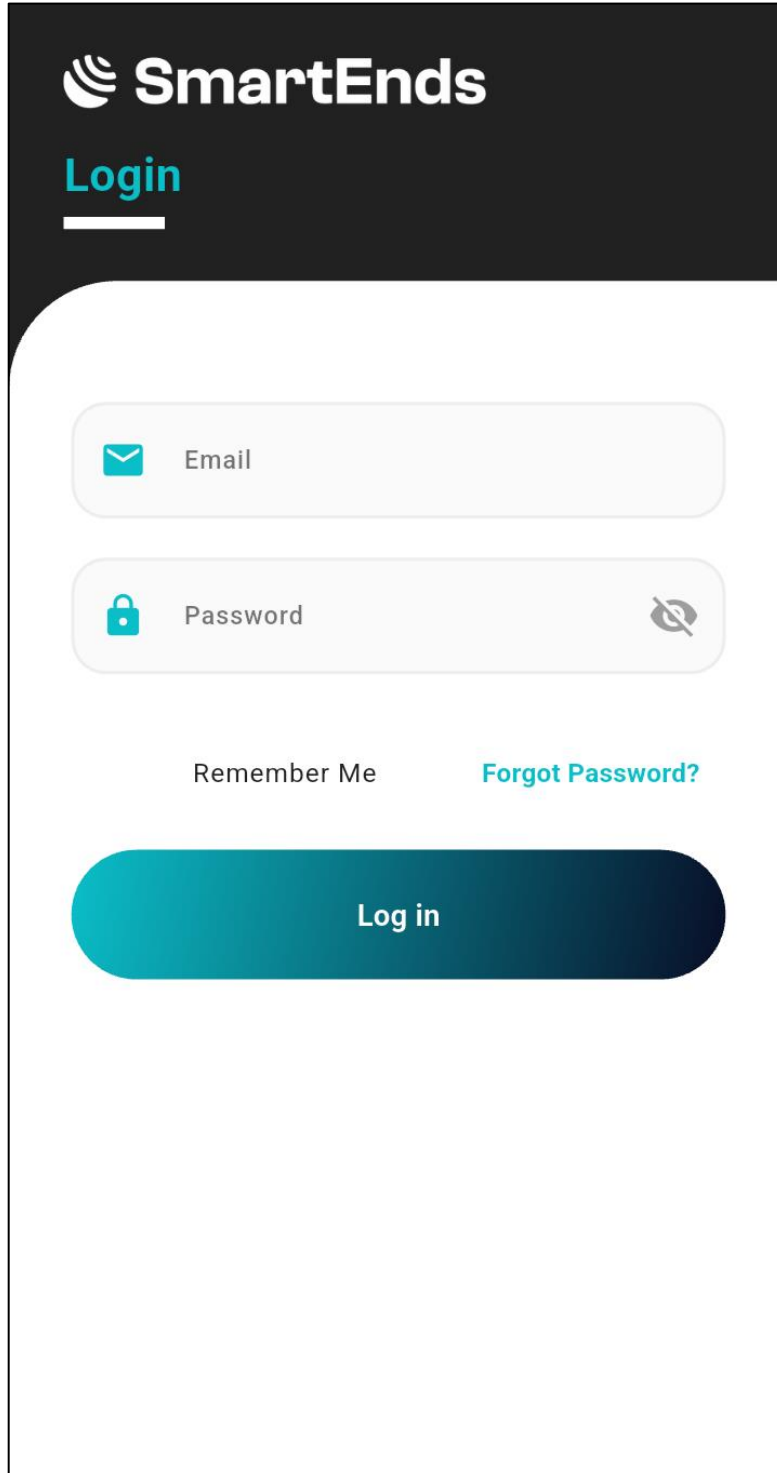
Step 7: End User Installation (APK)

- Download the Clean Core APK file from the provided link
- Go to Settings → Security → Enable 'Install from Unknown Sources'
- Locate and tap the APK file to begin installation
- Review permissions (Location, Camera, Internet, Notifications) and tap Install
- Open the app and grant location permissions on first launch
- Register a new account or log in with existing credentials

6.4 Operating Manual

This section provides a step-by-step guide for using the Clean Core application. It covers all essential functionalities including account creation, login, viewing bin fill levels on the map, planning optimized routes, marking bins as collected, reporting anomalies, and managing the system through the admin panel. Each functionality is explained with clear instructions and screenshots to help users operate the application confidently and efficiently.

6.4.1 Creating an Account



The image shows a mobile application login screen for 'SmartEnds'. The header is dark with the 'SmartEnds' logo and a 'Login' link. Below the header is a white rounded rectangle containing the login form. The form has two input fields: 'Email' with an envelope icon and 'Password' with a lock icon and a toggle for visibility. There are links for 'Remember Me' and 'Forgot Password?'. A large 'Log in' button is at the bottom.

SmartEnds

[Login](#)

Email

Password 

[Remember Me](#) [Forgot Password?](#)

Log in

Figure 6.1: Creating Account

6.4.2 Logging In

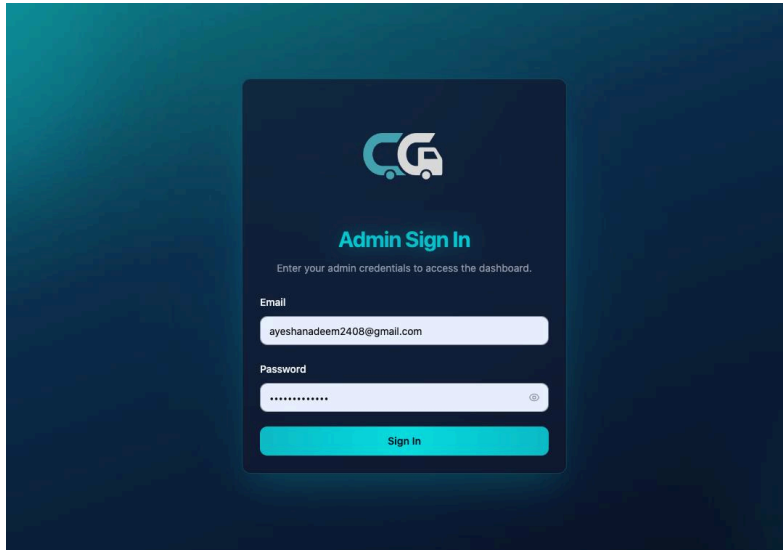


Figure 6.2: Logging in Admin



Figure 6.3: Logging in driver

6.4.3 Viewing Bins on Map

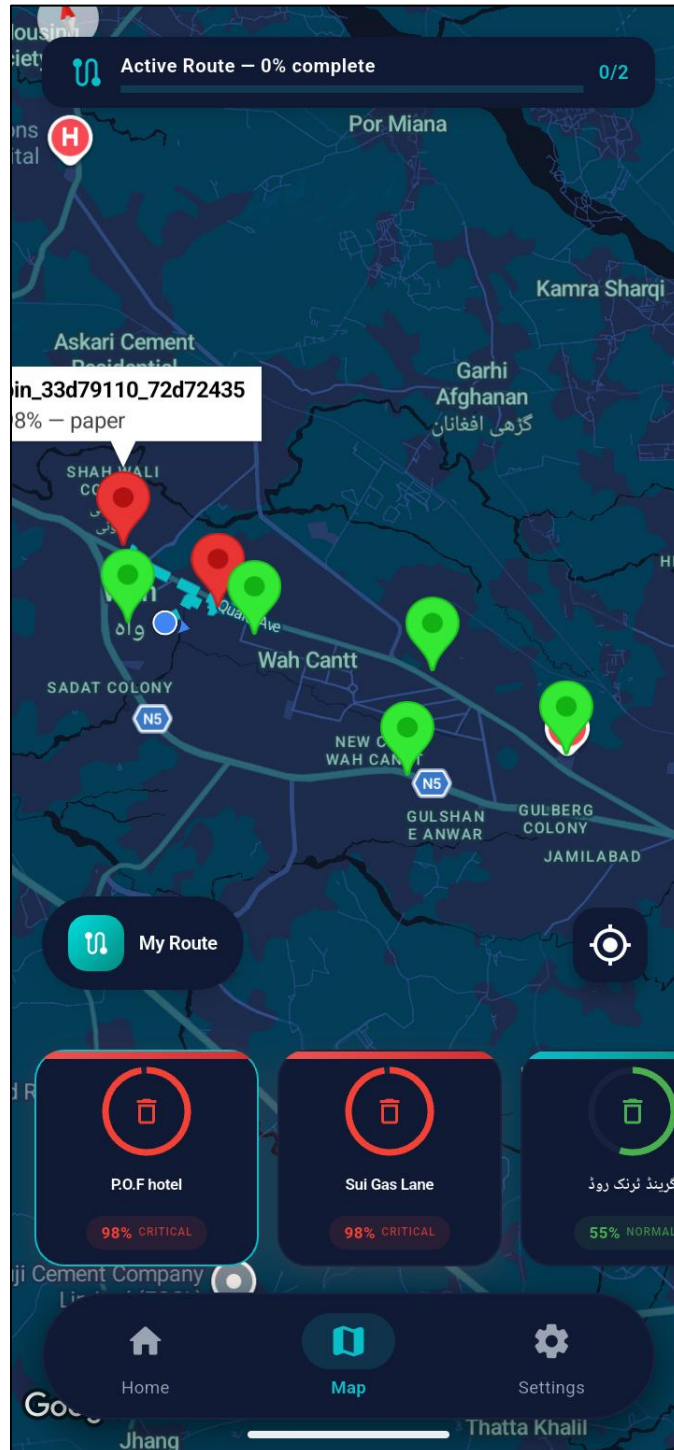


Figure 6.4: Viewing Bins on Map

6.4.4 Planning an Optimized Route

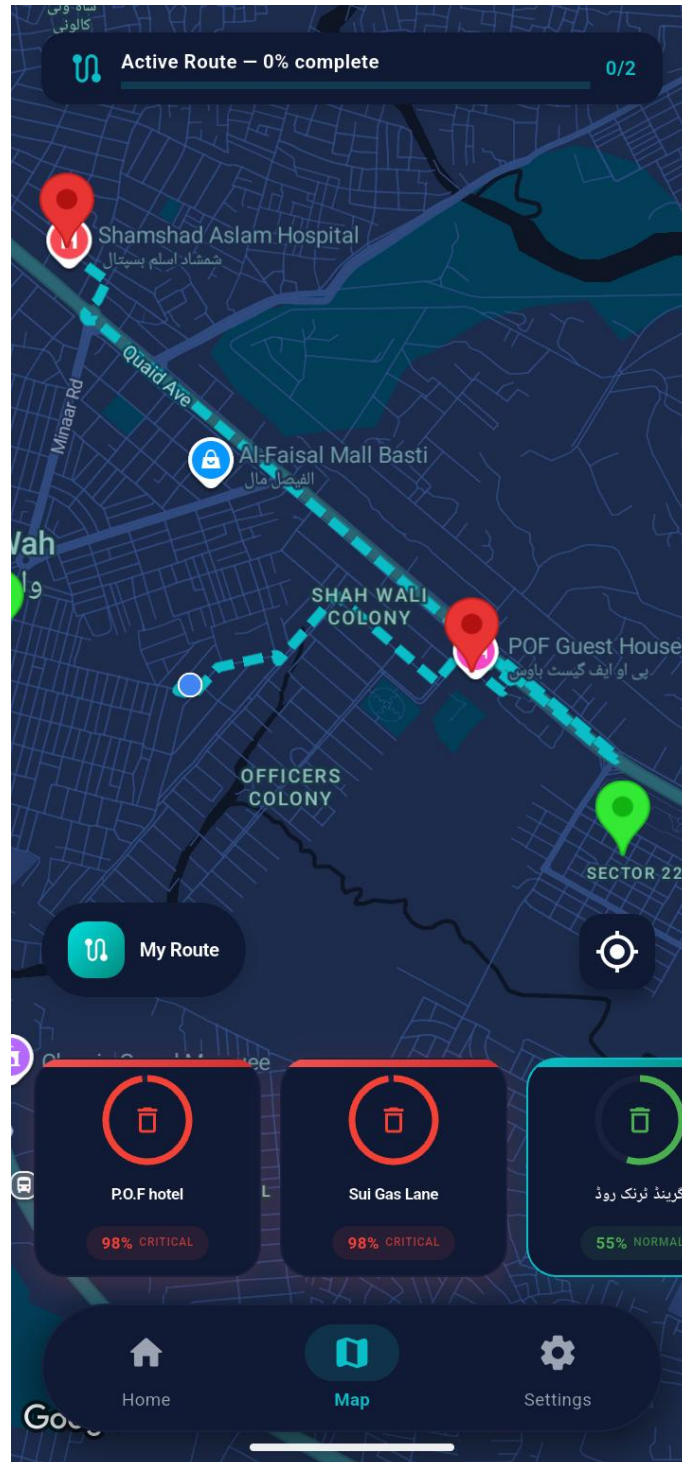


Figure 6.5: Routing

6.4.5 Marking Bins as Collected

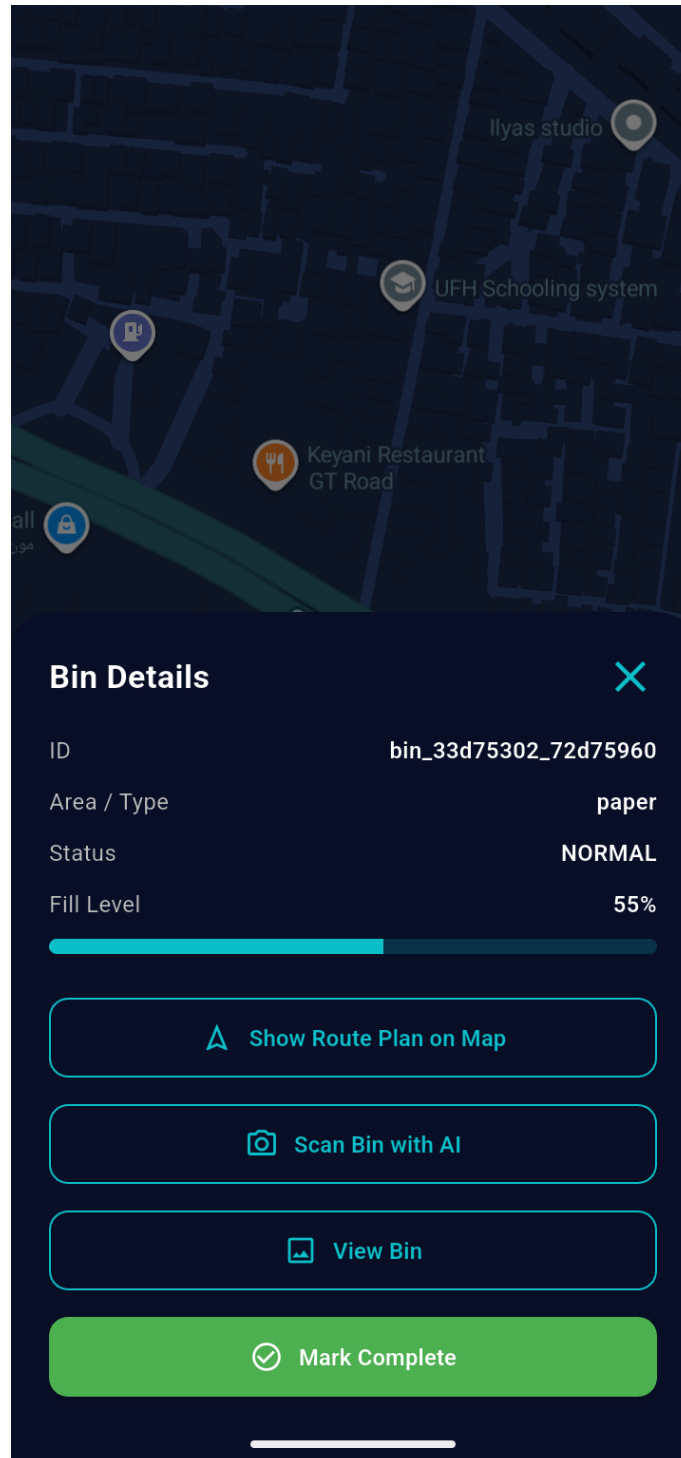


Figure 6.6: Marking bin Complete by Driver

6.4.6 Managing Profile

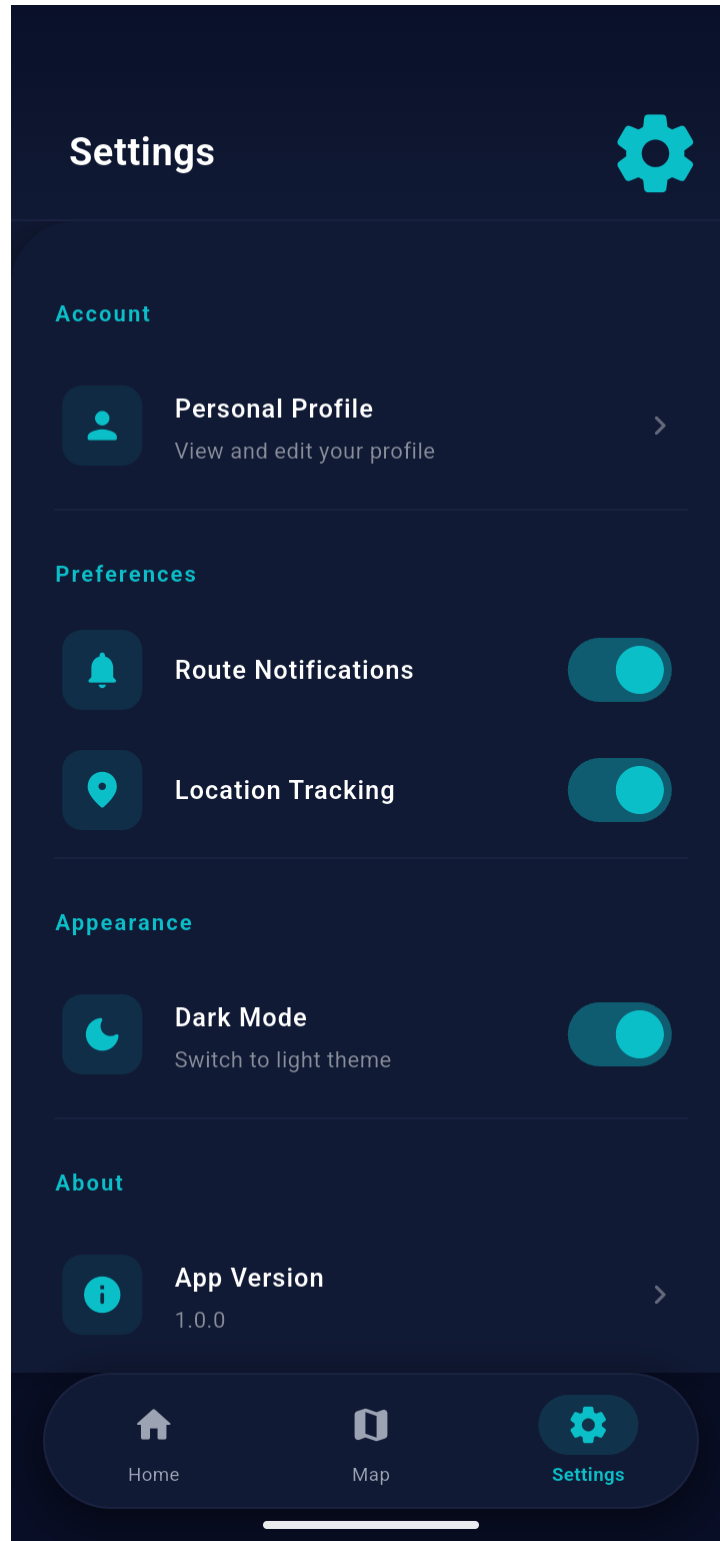


Figure 6.7: Updating Profile

6.4.7 Admin Panel Operations (Web)

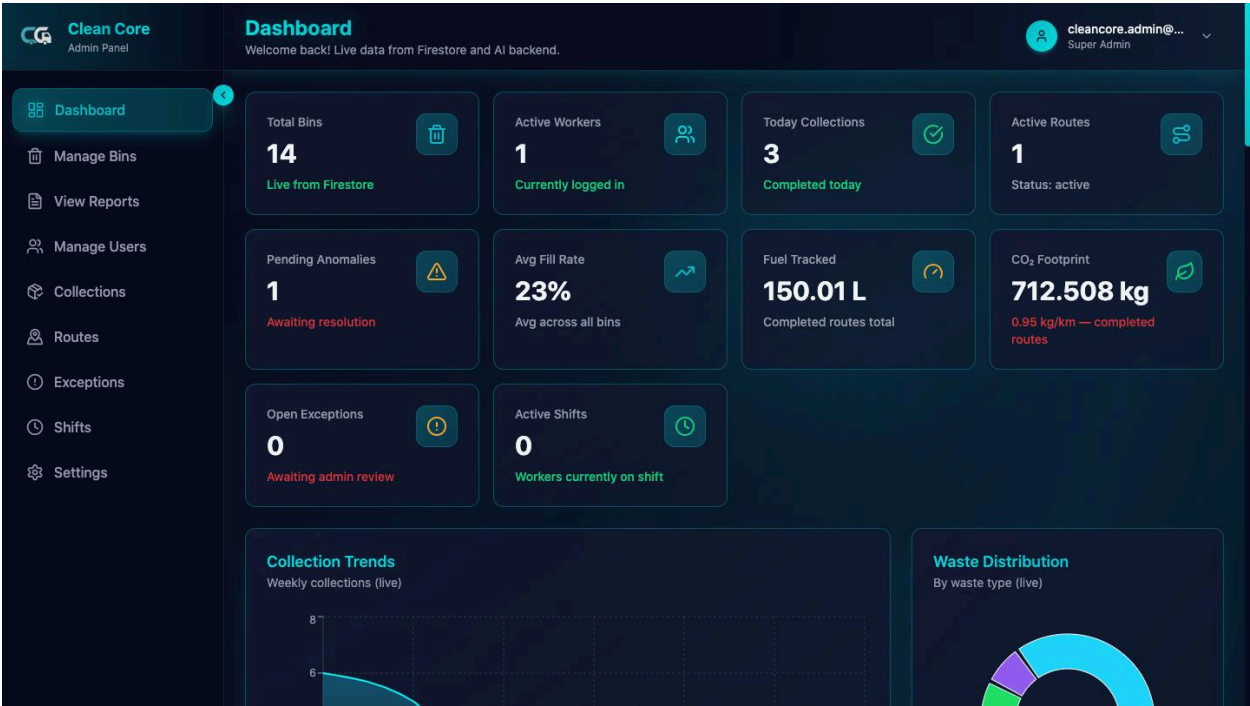


Figure 6.8: Admin Portal

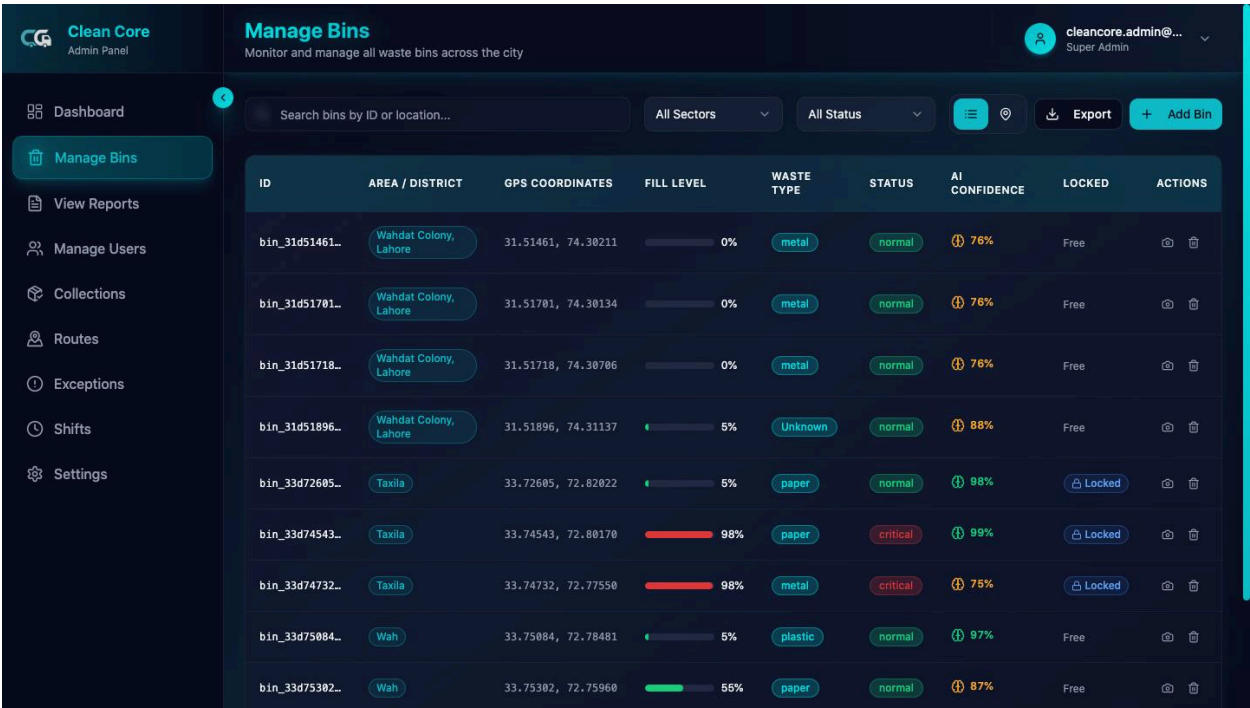


Figure 6.9: Manage Bins

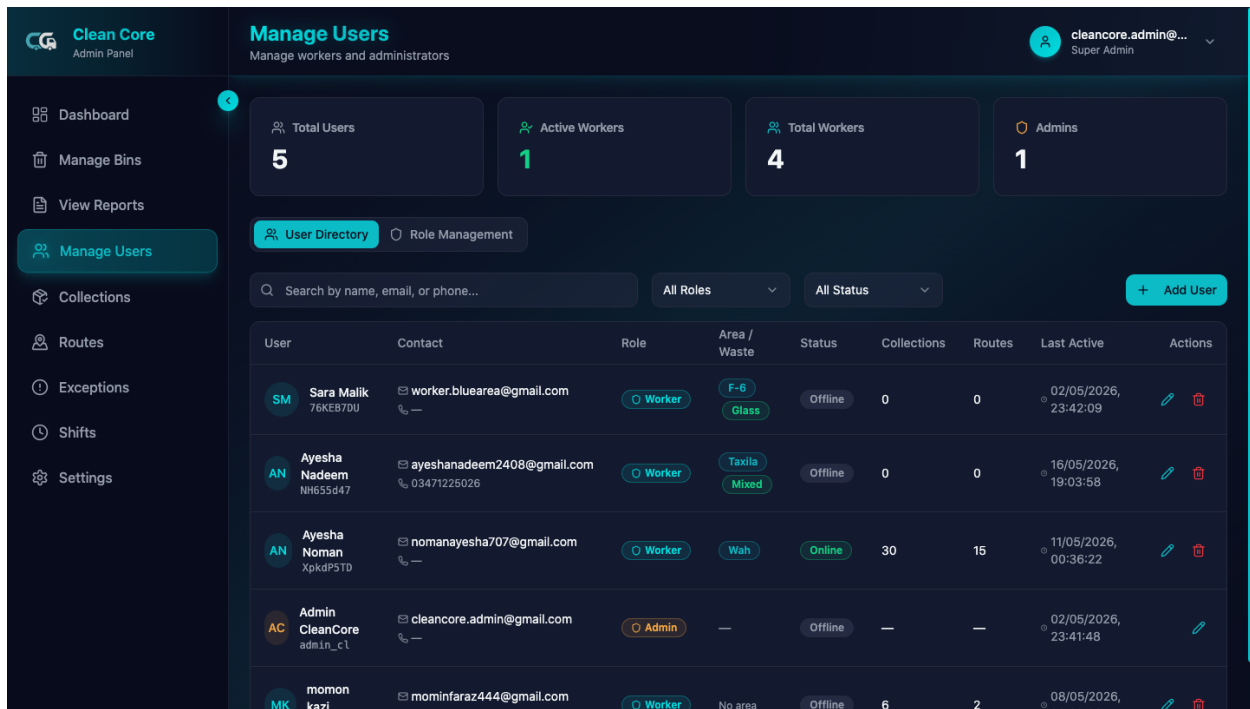


Figure 6.10: Manage Users

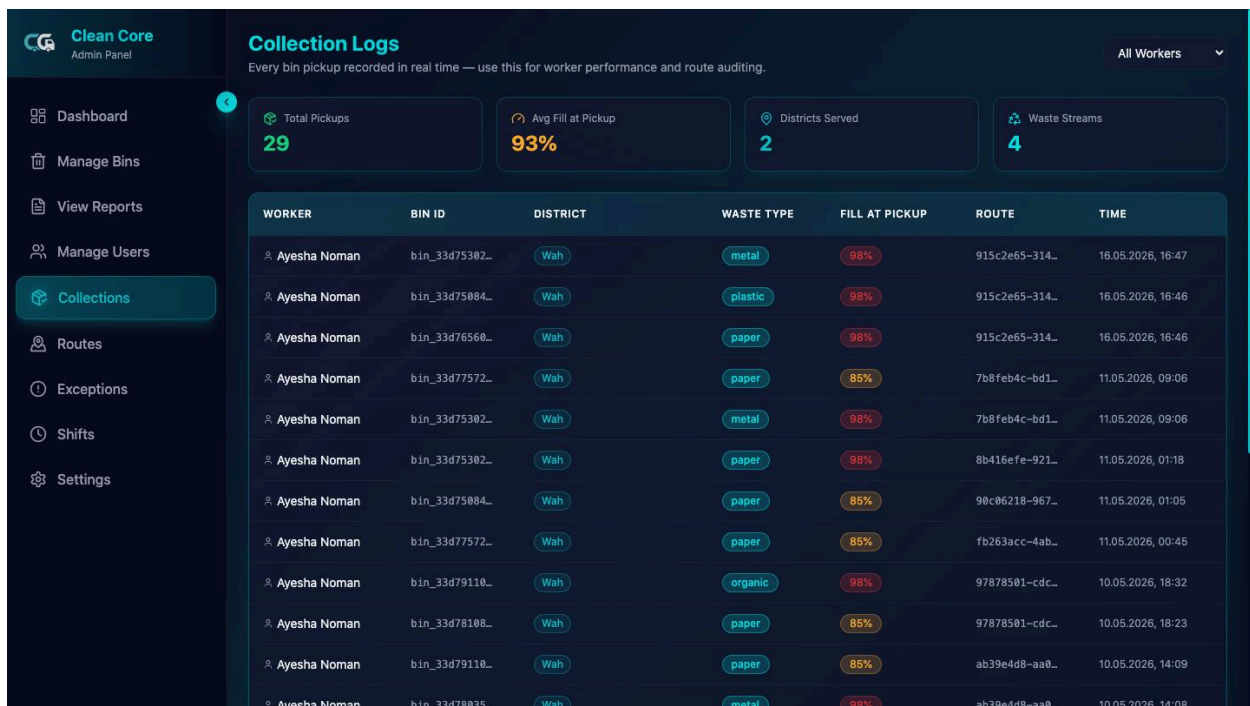


Figure 6.11: Collection Logs